University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab tasks	**Dated: 16 Feb 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 1**
Lab Title: Input/Output, Data Types, Operators, Conditional Statements (If/Else)	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Week 1 LAB

Input/Output, Data Types, Operators, Conditional Statements (If/Else)

Lab Goals/Objectives:

* Learn Python syntax.

* Take user input and display output.

* Understand data types and operators.

* Implement decision making using if/else.

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

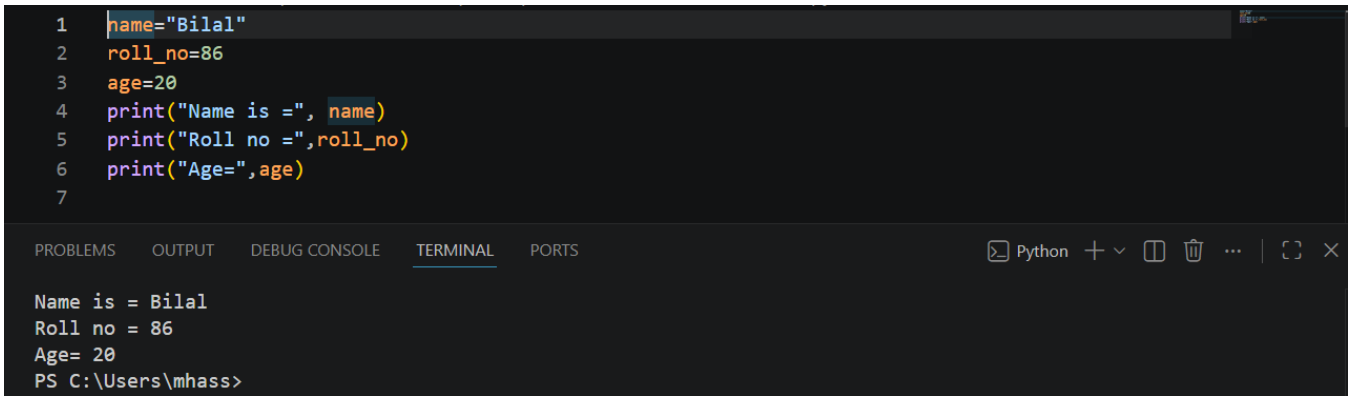
Python provides the input() function for taking input and print() for displaying output. Variables store different data types such as integers, floats, strings, and booleans. Operators perform arithmetic, logical, and comparison operations. Conditional statements (if, elif, else) execute different blocks based on conditions.

Lab Work:

Task 1: Initializes basic variables with specific values and prints them out.

name = "Bilal"
roll_no = 86
age = 20

Result:



Task 2: This task collects a user's name, roll number, and age dynamically via console inputs and displays them.

a=input("Enter your name=")
b=input("Enter your Roll no=")
c=input("Enter your age=")

Result:

```
a=input("Enter your name=")
b=input("Enter your Roll no=")
c=input("Enter your age=")
print(a)
print(b)
print(c)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                Python

Enter your name=Bilal
Enter your Roll no=86
Enter your age=20
Bilal
86
20
PS C:\Users\mhass>
```

Task 3: Adds two integers together and outputs the result.

a=10
b=12
sum=a+b

Result:

```
a=10
b=12
sum=a+b
print(sum)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                Python

22
PS C:\Users\mhass>
```

Task 4: Type casting by converting string digits into integers before adding them.

a=int("1")
b=int("2")
sum=a+b

Result:

```
a=int("1")
b=int("2")
sum=a+b
print(sum)
```

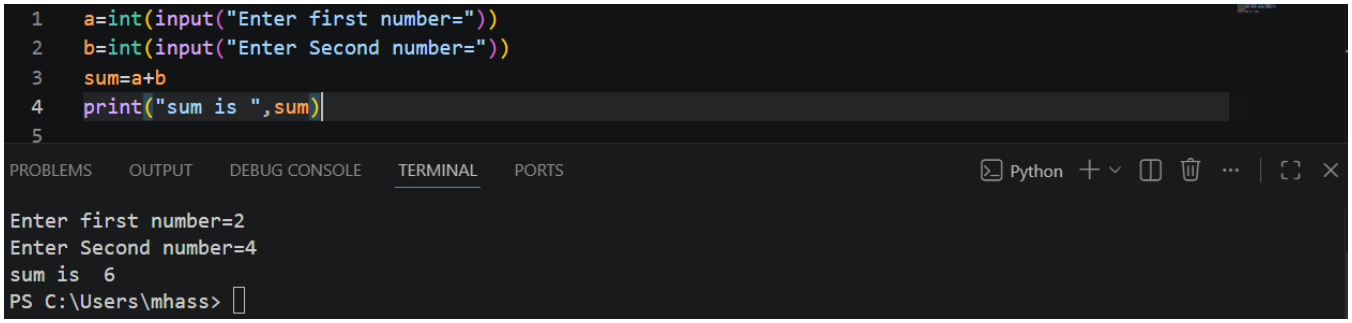
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                Python

3
PS C:\Users\mhass>
```

Task 5: Combines user input with integer casting to calculate the sum of two numbers provided by the user.

```
a=int(input("Enter first number"))
b=int(input("Enter Second number"))
sum=a+b
```

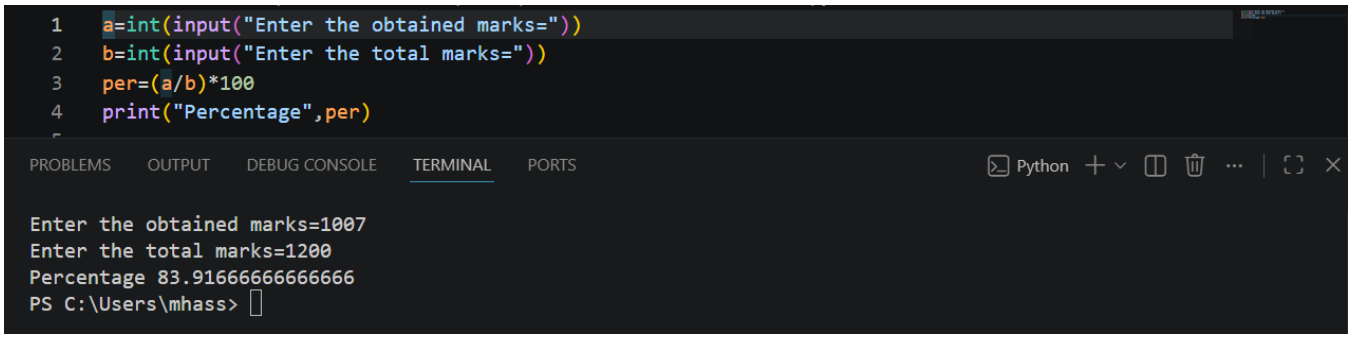
Result:



Task 6: Calculates the percentage based on user-inputted obtained and total marks.

```
per=(a/b)*100
```

Result:



Task 7: Extends the percentage calculator by using conditional logic (`if-elif-else`) to assign a letter grade.

```
if(per>=90):
        print("Grade A")

elif(per>=85):
        print("Grade A-")

elif(per>=80):
        print("Grade B+")

elif(per>=75):
        print("Grade B-")

elif(per>=70):
        print("Grade C")

else :
        print("Fail")
```

Result:

```
a=int(input("Enter the obtained marks="))
b=int(input("Enter the total marks="))
per=(a/b)*100
print("Percentage",per)
if(per>=90):
    print("Grade A")
elif(per>=85):
    print("Grade A-")
elif(per>=80):
    print("Grade B+")
elif(per>=75):
    print("Grade B-")
elif(per>=70):
    print("Grade C")
else :
    print("Fail")
```

```
Enter the obtained marks=1007
Enter the total marks=1200
Percentage 83.91666666666666
Grade B+
PS C:\Users\mhass>
```

Task 8: Uses a `for` loop to calculate percentages and grades for a fixed, user-defined number of students.

```
c=int(input("Enter the total Students numbers="))
for i in range(c):
    if(per>=90):
            g="A"
    elif(per>=85):
            g="A-"
    elif(per>=80):
            g="B+"
    elif(per>=75):
            g="B-"
    elif(per>=70):
            g="C"
    else :
            g="F"
```

Result:

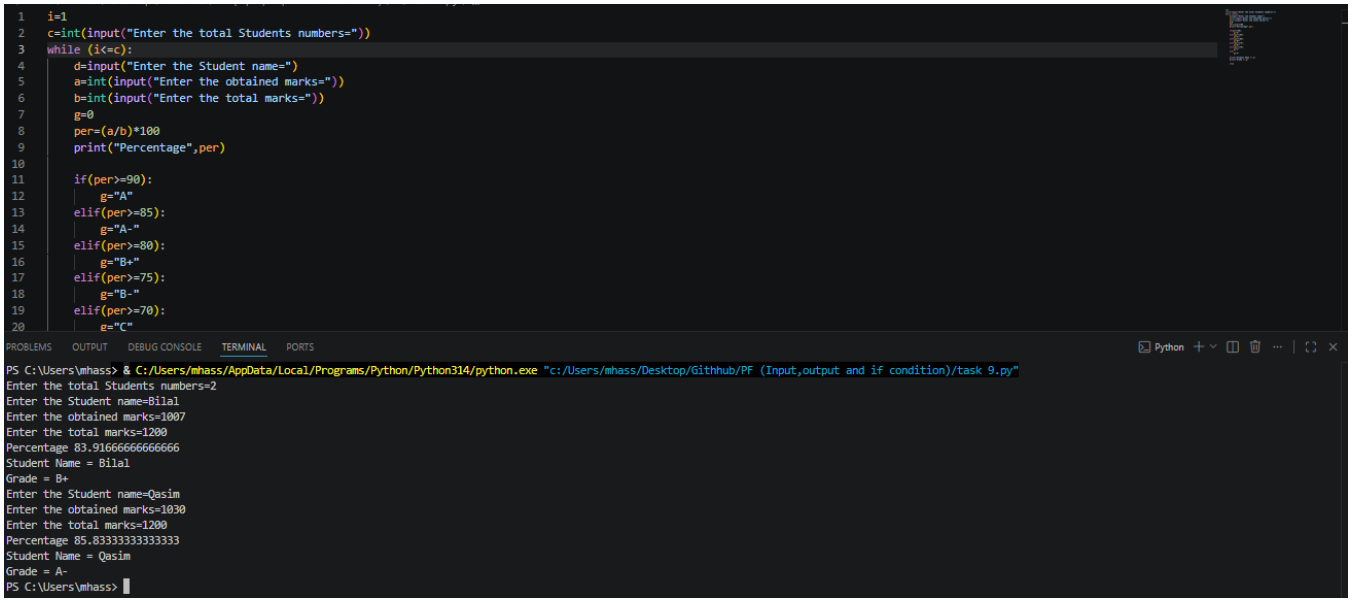
```
c=int(input("Enter the total Students numbers="))
for i in range(c):
    d=input("Enter the Student name=")
    b=int(input("Enter the total marks="))
    a=int(input("Enter the obtained marks="))
    g=0
    per=(a/b)*100
    print("Percentage",per)
    if(per>=90):
        g="A"
    elif(per>=85):
        g="A-"
    elif(per>=80):
```

```
Enter the total Students numbers=3
Enter the Student name=Bilal
Enter the total marks=1007
Enter the obtained marks=1200
Percentage 119.16583912611718
Student Name = Bilal
Grade = A
Enter the Student name=Qasim
Enter the total marks=1000
Enter the obtained marks=1200
Percentage 120.0
Student Name = Qasim
Grade = A
Enter the Student name=Shaheer
Enter the total marks=1100
Enter the obtained marks=1200
Percentage 109.09090909090908
Student Name = Shaheer
Grade = A
PS C:\Users\mhass>
```

Task 9: Achieves the same multi-student grading functionality as Task 8, but implements a `while` loop control structure instead.

```
i=1
c=int(input("Enter the total Students numbers="))
while (i<=c):
        ###if elif conditions for Grading
```

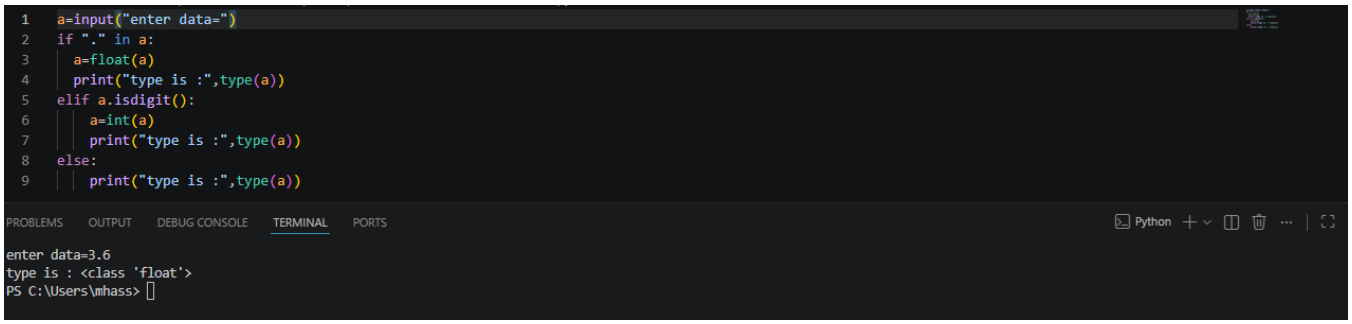
Result:



Task 10: Analyzes a raw text input to determine and cast it into its most appropriate data type (`float`, `int`, or default `str`).

```
if "." in a:
   a=float(a)
elif a.isdigit():
   a=int(a)
```

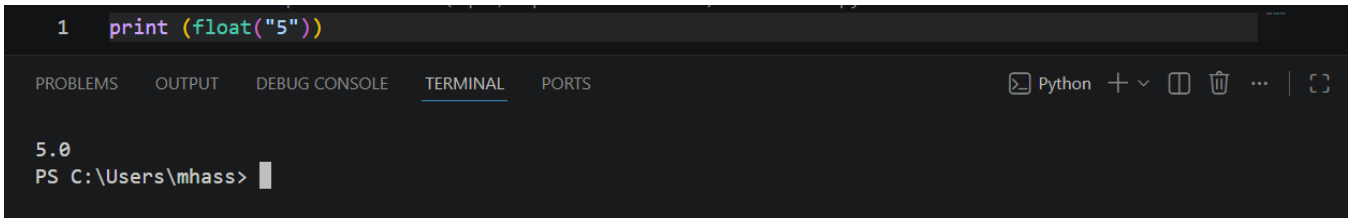
Result:



Task 12: Print integer as float.

```
print (float("5"))
```

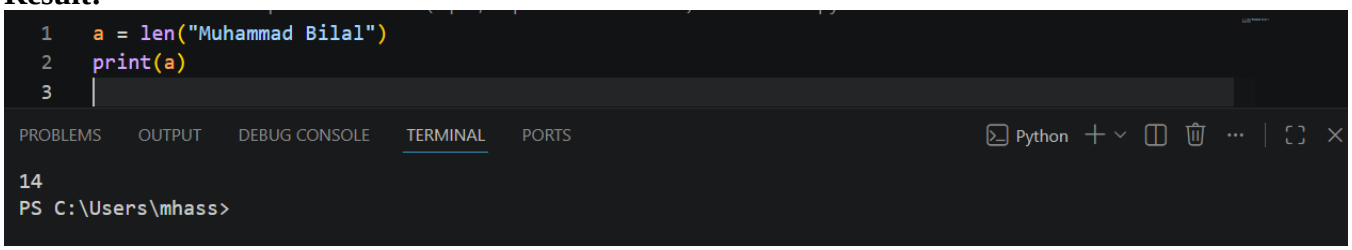
Result:



Task 13: Print the length of string.

```
a = len("Muhammad Bilal")
```

Result:



Conclusions:

These exercises show how to take raw data, convert it accurately, apply logical conditions, and automate the entire workflow to build responsive and efficient scripts.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab tasks	**Dated: 2 March 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 1**
Lab Title: For Loops	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Week 2 LAB

For Loops

Lab Goals/Objectives:

- To understand and implement **definite iteration** using Python's `for` loop.
- To learn how to iterate over sequences (like ranges and lists) and manipulate data during iteration.
- To apply conditional logic (`if` statements) inside loops for selective data processing.

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

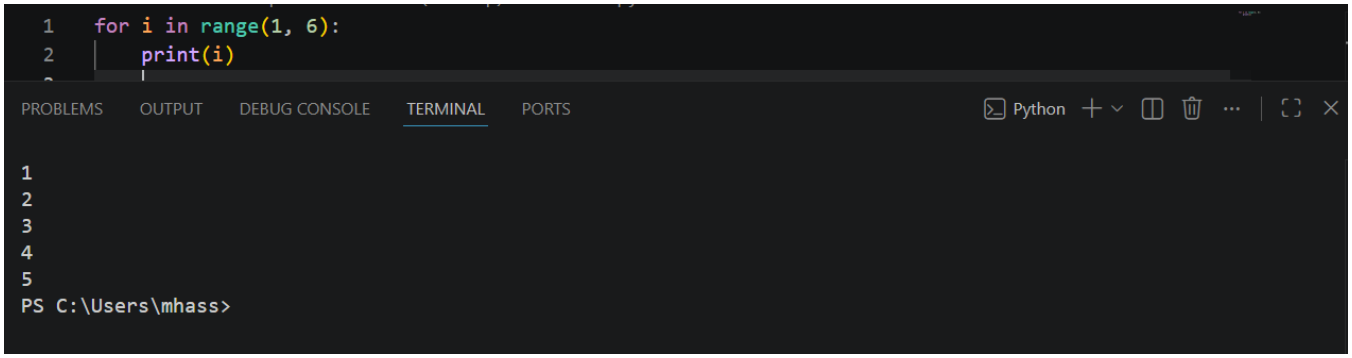
In Python, a `for` loop is used to iterate over a sequence (such as a list, tuple, dictionary, set, or string) or a range of numbers. Unlike some other languages, Python's `for` loop acts more like an iterator method

Lab Work:

Task 1: Basic Range Iteration

```
for i in range(1, 6):
    print(i)
```

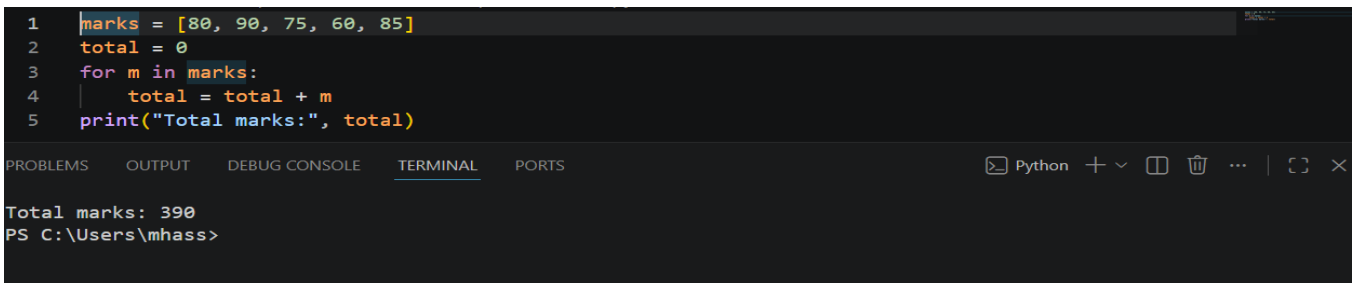
Result:



Task 2: List Iteration and Accumulation

```
marks = [80, 90, 75, 60, 85]
for m in marks:
    total = total + m
```

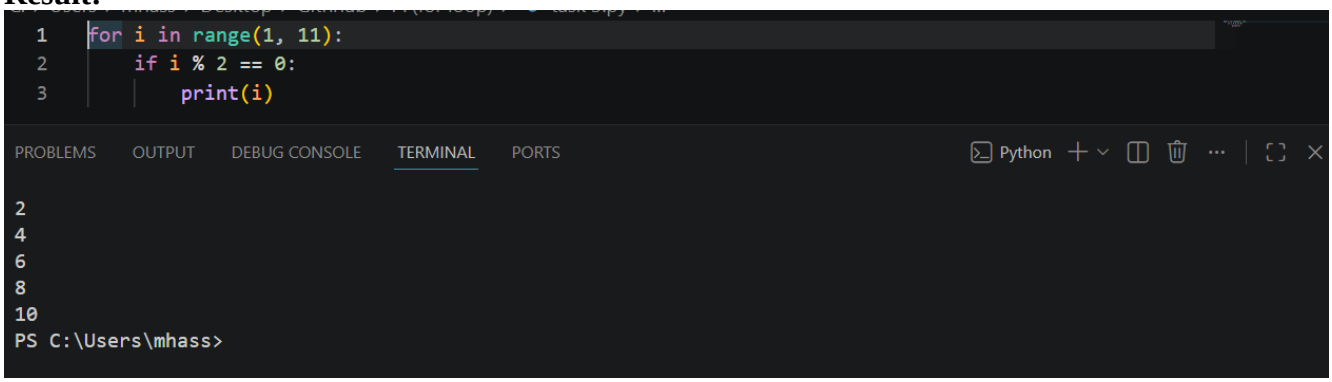
Result:



Task 3: Conditional Logic within a Loop.

```
for i in range(1, 11):
    if i % 2 == 0:
        print(i)
```

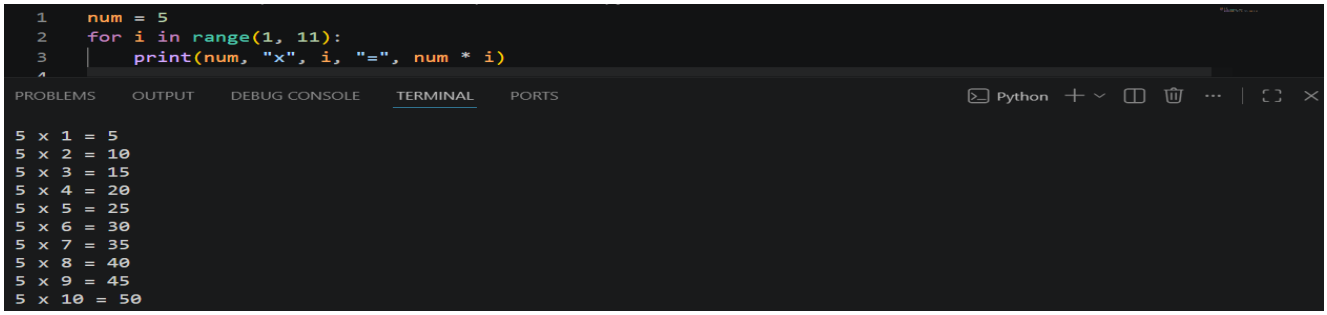
Result:



Task 4: Multiplication Table Generation.

```
for i in range(1, 11):
    print(num, "x", i, "=", num * i)
```

Result:



Conclusions:

This lab successfully demonstrates the utility of **`for` loops** in Python for handling repetitive tasks efficiently. By practicing with ranges, lists, and conditional statements, we covered the core fundamentals of iteration, data accumulation, and basic logic control. These building blocks are essential for writing clean, automated, and scalable code.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab tasks	**Dated: 16 March 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 1**
Lab Title: While Loops	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Week 3 LAB

While Loops

Lab Goals/Objectives:

- To understand and implement **indefinite and conditional iteration** using Python's `while` loop.
- To learn how to manage loop control variables through manual increments, decrements, and user input.
- To master the **sentinel value** pattern to dynamically terminate a loop based on user interaction.

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

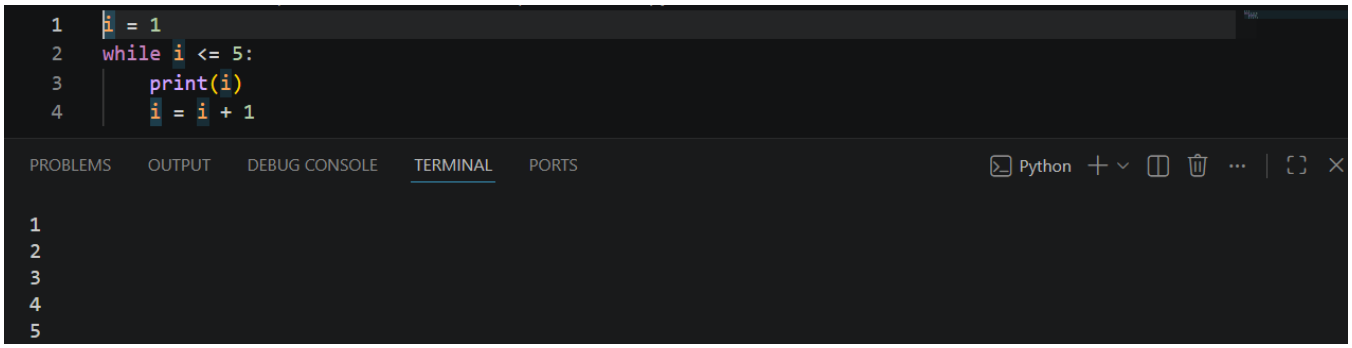
Unlike a `for` loop which iterates over a fixed sequence, a `while` loop in Python repeatedly executes a block of code as long as a given condition remains `True`.

Lab Work:

Task 1: Basic While Loop Iteration (Counting Up).

```
while i <= 5:
  print(i)
 i = i + 1
```

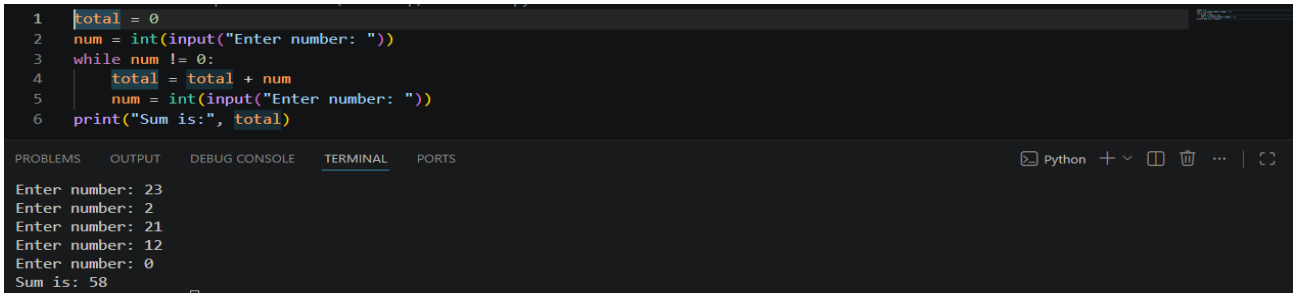
Result:



Task 2: Sentinel-Controlled Loop (Accumulation)

```
while num != 0:
  total = total + num
  num = int(input("Enter number: "))
```

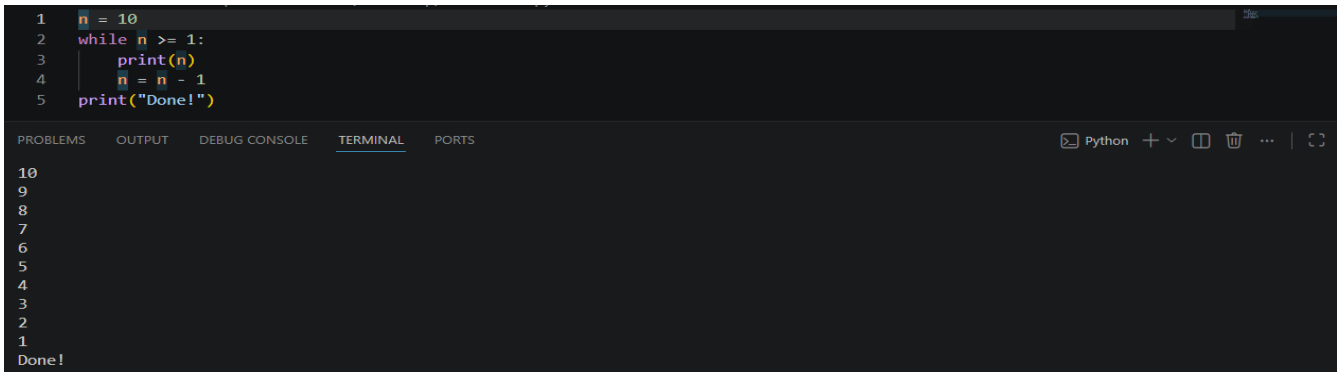
Result:



Task 3: Countdown Iteration

```
while n >= 1:
    print(n)
    n = n - 1
```

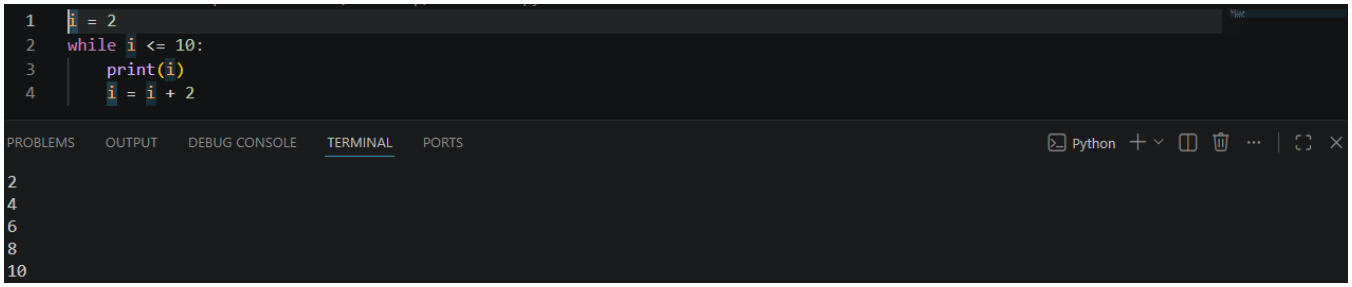
Result:



Task 4: Iterating with Step Increments (Even Numbers)

```
while i <= 10:
  print(i)
  i = i + 2
```

Result:



Conclusions:

This lab successfully demonstrates how **`while` loops** function in Python to handle condition-based repetition. By implementing countdowns, variable step increases, and sentinel values, we observed how `while` loops provide flexible control for scenarios where the exact number of iterations is unknown beforehand.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab tasks	**Dated: 23 March 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 1**
Lab Title: Nested Loops	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Week 4 LAB

Nested Loops

Lab Goals/Objectives:

- To understand and implement **nested loops** (loops inside loops) using both `for` and `while` structures in Python.

- To master coordinate tracking and grid-based logic using outer loop (`i`) and inner loop (`j`) variables.

- To generate complex geometric text patterns, pyramids, and inverted shapes through algorithmic whitespace and character printing.

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

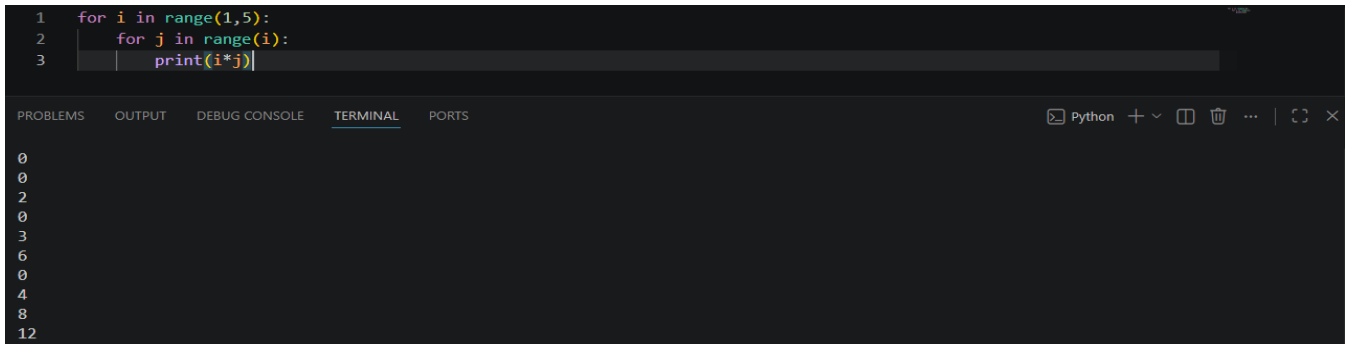
Nested loops consist of an **outer loop** and one or more **inner loops**. Each time the outer loop runs once, the inner loop executes completely through all of its iterations.

Lab Work:

Task 1: Multiplication Matrix Generation

```
for i in range(1, 5):
    for j in range(i):
        print(i * j)
```

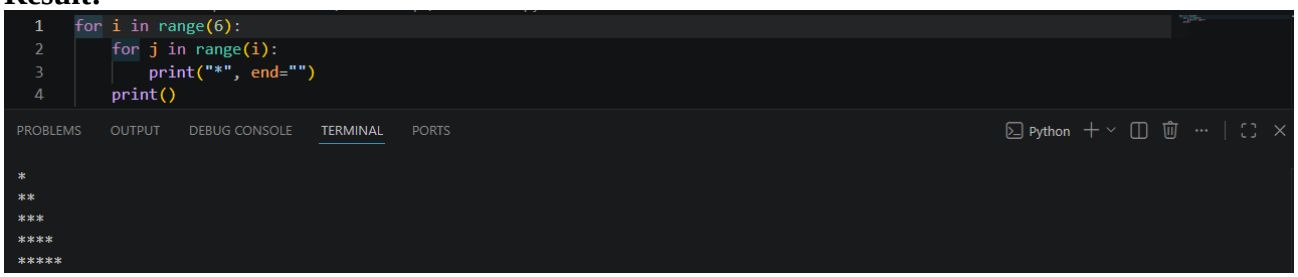
Result:



Task 2: Standard Left-Aligned Triangle Pattern

```
for i in range(6):
    for j in range(i):
        print("*", end="")
    print()
```

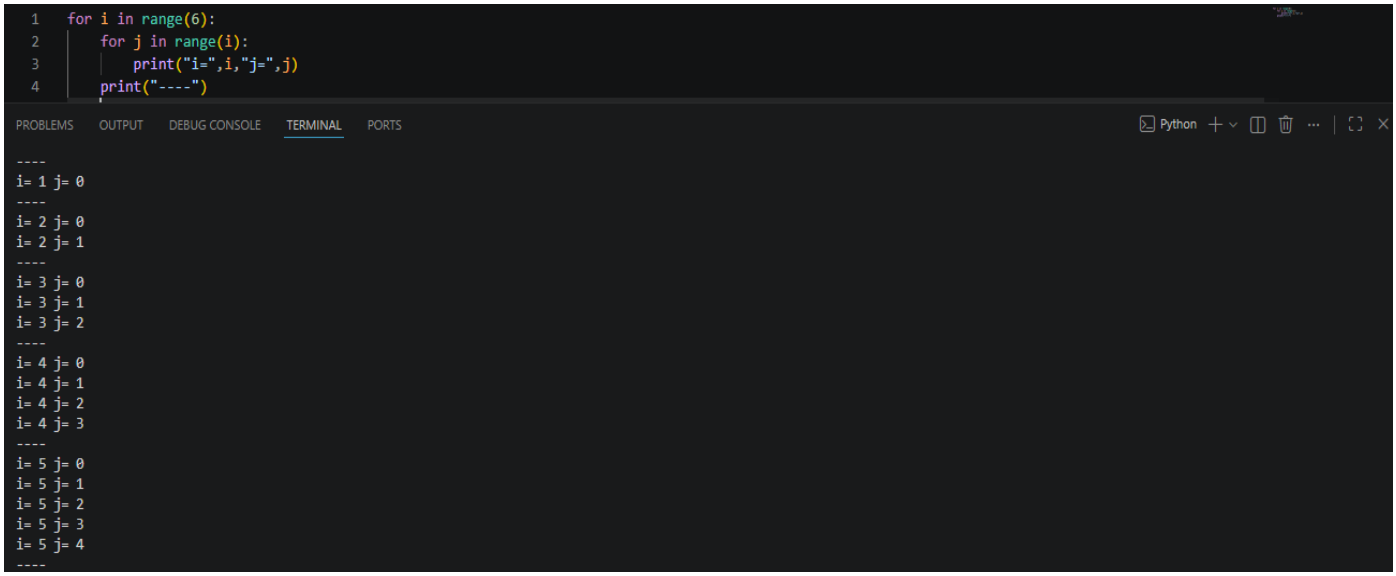
Result:



Task 3: Nested Loop Coordinate Tracking (Rows and Columns)

```
for i in range(6):
  for j in range(i):
    print("i=", i, "j=", j)
  print("----")
```

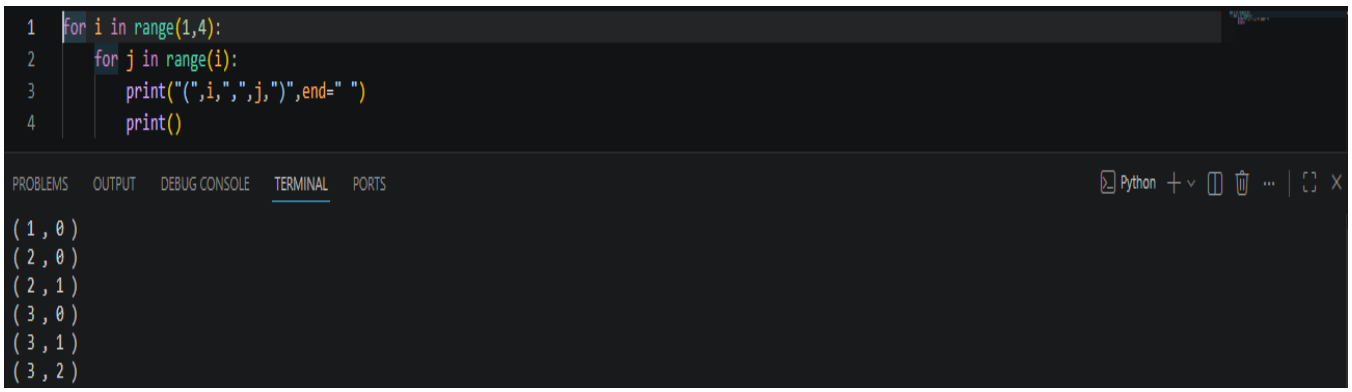
Result:



Task 4: Explicit Tuple Mapping (Vertical Layout)

```
for i in range(1, 4):
  for j in range(i):
    print("(", i, ",", j, ")", end=" ")
  print()
```

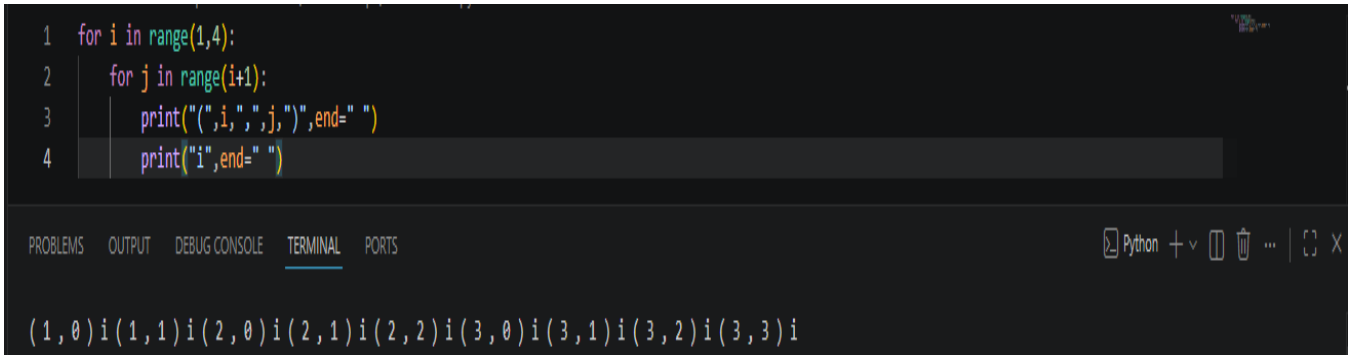
Result:



Task 5: Dynamic Row-Column Boundaries (Offset Matrix)

```
for i in range(1, 4):
    for j in range(i + 1):
        print("(", i, ",", j, ")", end=" ")
    print("i", end=" ")
```

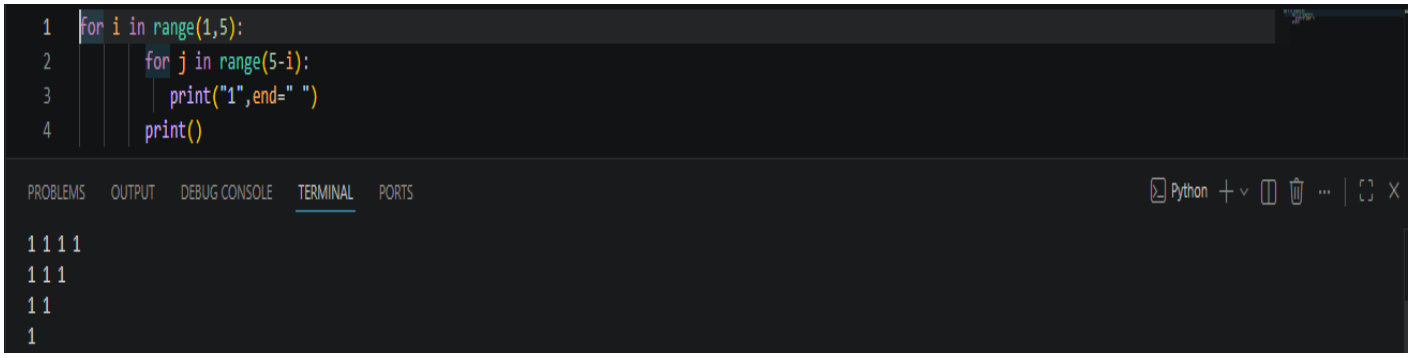
Result:



Task 6: Inverted Inverted Number Block (Shrinking Rows)

```
for i in range(1, 5):
  for j in range(5 - i):
        print("1", end=" ")
  print()
```

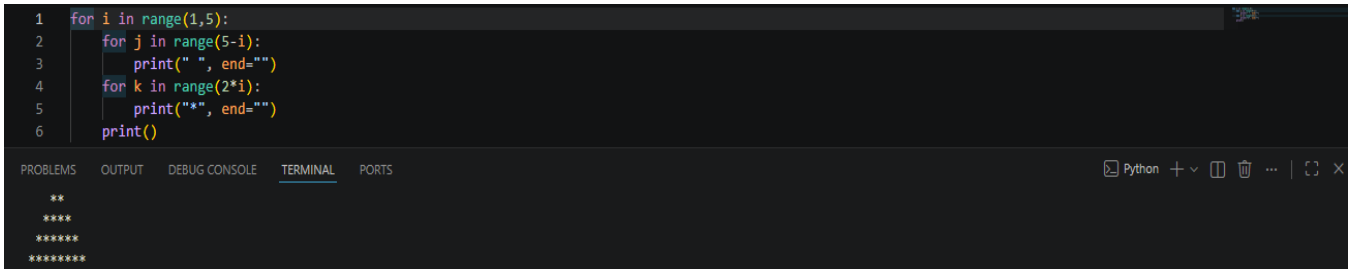
Result:



Task 7: Symmetrical Star Pyramid (`for` loop approach)

```
for i in range(1, 5):
    for j in range(5 - i):
        print(" ", end="")
    for k in range(2 * i):
        print("*", end="")
    print()
```

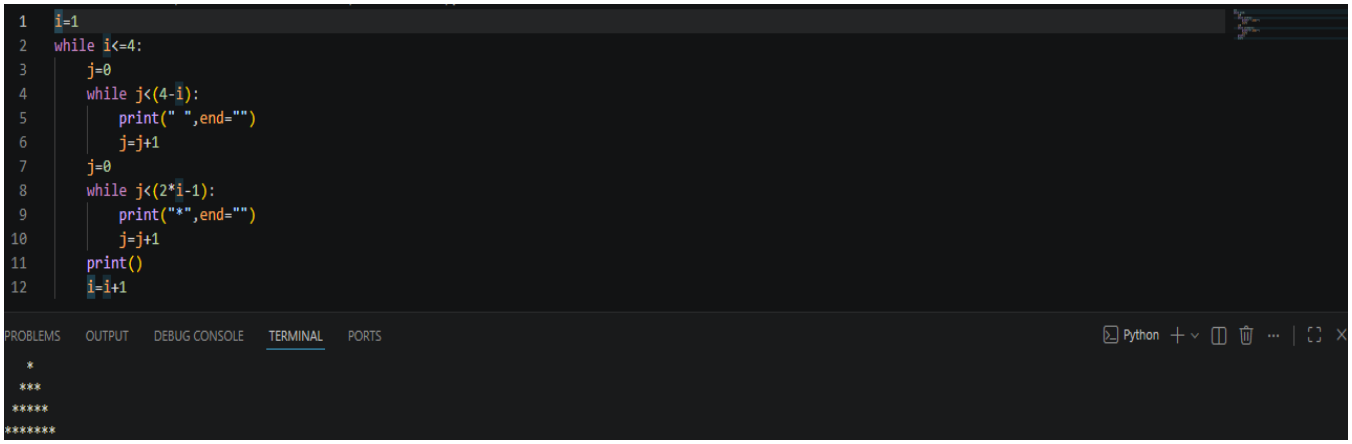
Result:



Task 8: Symmetrical Star Pyramid (`while` loop approach)

```
while i <= 4:
    while j < (4 - i):
        print(" ", end="")
        j = j + 1
    while j < (2 * i - 1):
        print("*", end="")
        j = j + 1
    i = i + 1
```

Result:



Conclusions:

This lab successfully demonstrates how **nested iteration** controls multidimensional outputs. By manipulating the execution bounds of an inner loop relative to its outer loop, we can systematically plot mathematical grids, track strict index steps, and calculate whitespace geometry to build both simple blocks and complex centered pyramids.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab tasks	**Dated: 30 March 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 2**
Lab Title: Functions	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO2	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Week 5 LAB

Functions

Lab Goals/Objectives:

- To understand the definition, creation, and execution of **functions** in Python.
- To learn how to pass data into functions using **parameters and arguments**.
- To master the difference between functions that perform actions (`print()`) and functions that compute and pass back data (`return`).

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

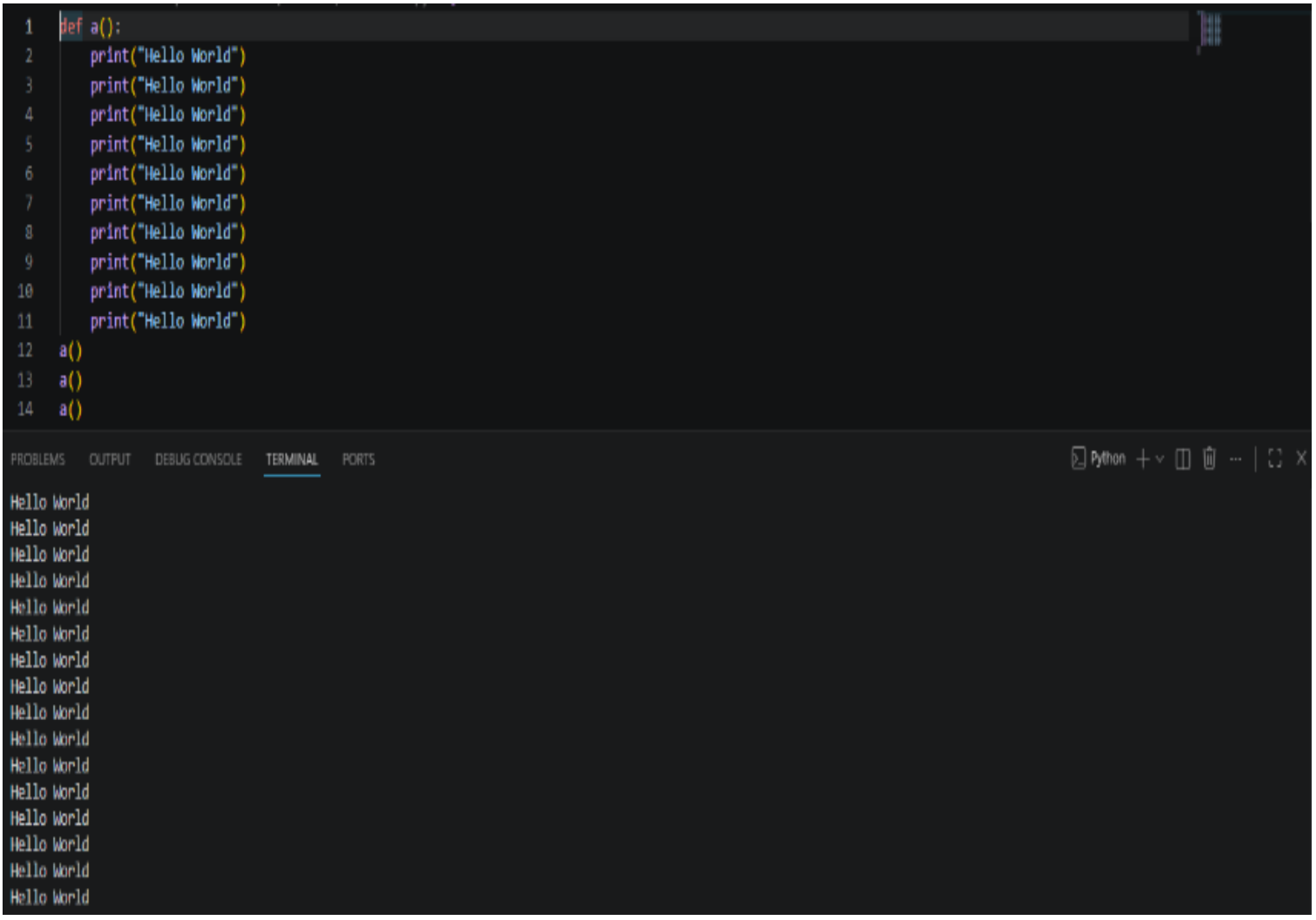
Functions are modular, reusable blocks of code designed to perform a specific task. They help break down complex programs into smaller, manageable, and organized pieces.

Lab Work:

Task 1: Basic Function Definition and Reusability

```
def a():
    print("Hello World")
a()
```

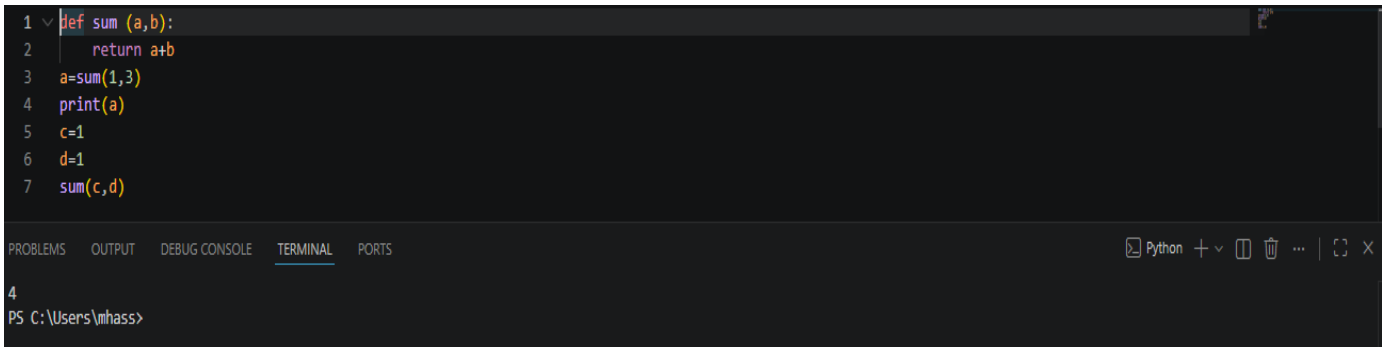
Result:



Task 2: Function with Parameters and Value Returning

```
def sum (a,b):
    return a+b
a=sum(1,3)
```

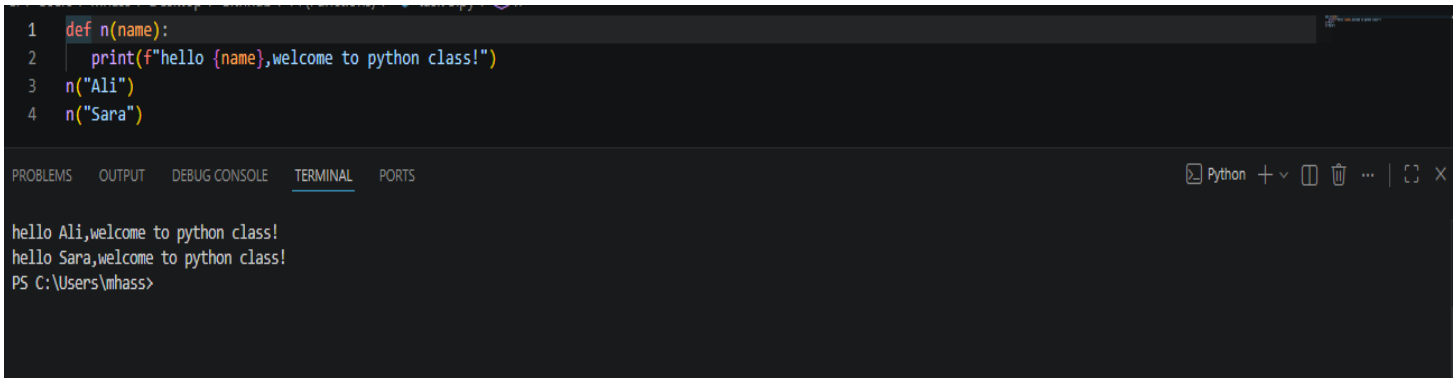
Result:



Task 3: Dynamic String Formatting with Arguments

```
def n(name):
    print(f"hello {name},welcome to python class!")
n("Ali")
```

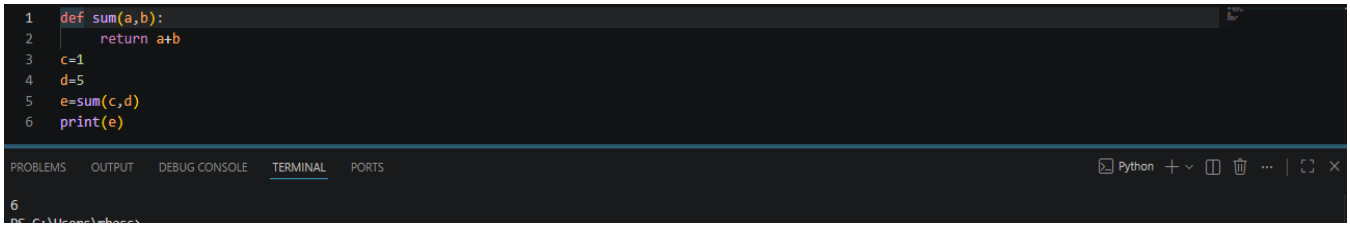
Result:



Task 4: Variable Assignment from Function Returns

```
def sum(a,b):
    return a+b
e=sum(c,d)
```

Result:



Conclusions:

This lab successfully demonstrates the foundational concepts of **procedural abstraction** using Python functions. By encapsulating code into reusable blocks, passing dynamic data via inputs, and capturing outputs with the `return` statement, we established how to eliminate code redundancy and build structured, maintainable software components.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab tasks	**Dated: 20 April 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 2**
Lab Title: Local and Global Variables	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO2	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Week 6 LAB

Local and Global Variables

Lab Goals/Objectives:

- To understand the concepts of **variable scope** (local vs. global variables) in Python functions.
- To analyze how Python resolves variable names using the **LEGB (Local, Enclosing, Global, Built-in)** rule.
- To identify common scope-related runtime errors, such as the `UnboundLocalError`.

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

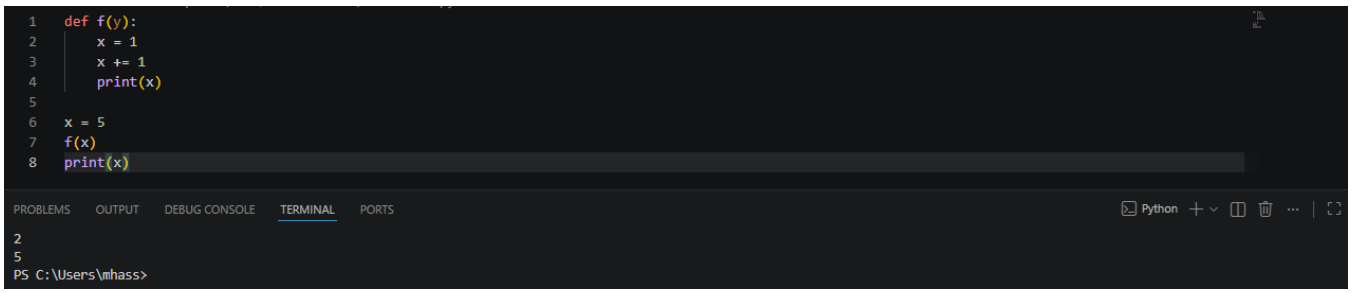
Scope determines the visibility and lifetime of a variable in a program. Where a variable is defined dictates where it can be accessed or modified.

Lab Work:

Task 1: Local vs. Global Variable Isolation

```
def f(y):
  x = 1
  x += 1
  x = 5
f(x)
```

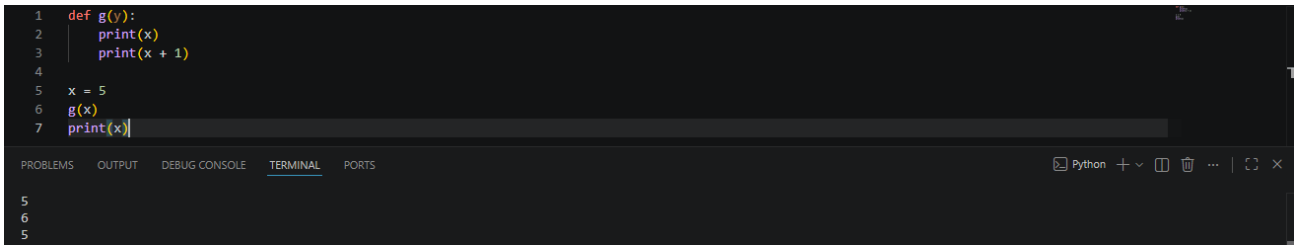
Result:



Task 2: Reading Global Variables Within a Function

```
def g(y):
  print(x)
  x = 5
g(x)
```

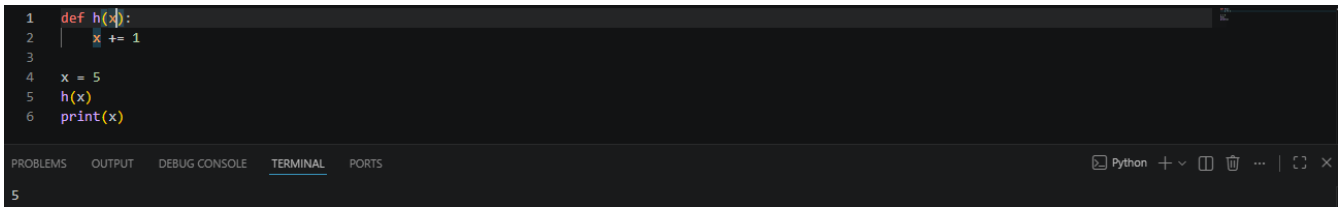
Result:



Task 3: UnboundLocalError Trigger (Illegal Global Modification)

```
def h(x):
    x += 1
x = 5
h(x)
```

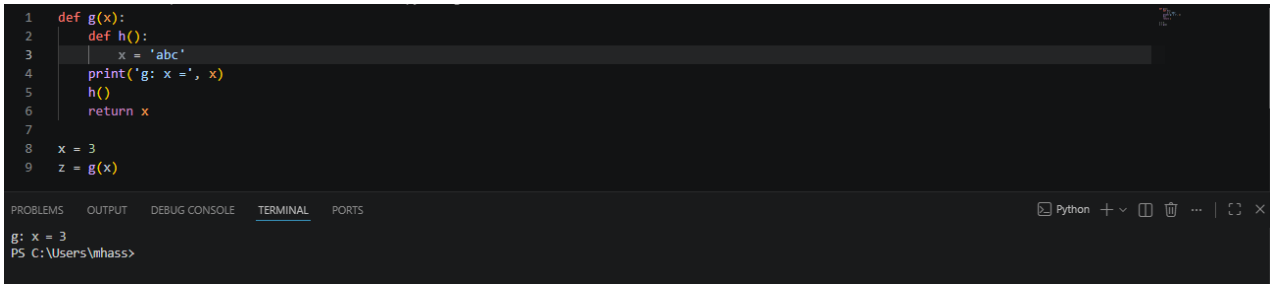
Result:



Task 4: Enclosing Scope and Nested Function Execution

```
def g(x):
    def h():
        x = 'abc'
    h()
    return x
```

Result:



Conclusions:

This lab successfully demonstrates the boundaries of **variable scoping rules** in Python. We observed that functions isolate their inner computations using local scopes, preventing accidental side effects on global variables. Furthermore, we explored how nested functions look outward to enclosing scopes, and how attempting to modify an outer variable without proper declaration triggers strict runtime scope errors.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab tasks	**Dated: 27 April 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 3**
Lab Title: List, Tuple	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO3	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Week 7 LAB

List, Tuple

Lab Goals/Objectives:

- To understand and manipulate **ordered sequences** in Python using lists and tuples.
- To master the concept of **mutability versus immutability** and how it dictates data modification.
- To practice item indexing, item extraction, and in-place list modification methods.

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

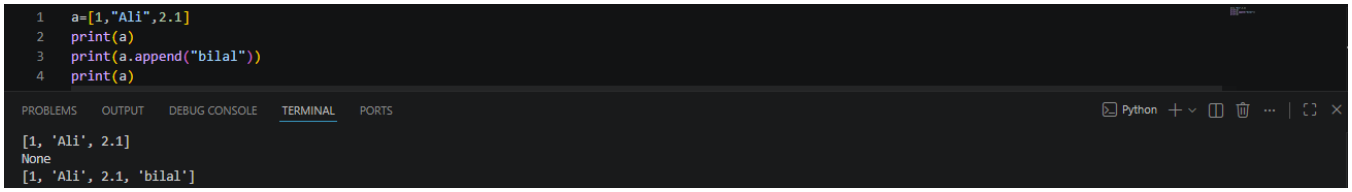
Python provides different built-in data types to store collections of items. Two of the most commonly used sequence types are **Lists** and **Tuples**.

Lab Work:

Task 1: List Creation and In-Place Appending

```
a = [1, "Ali", 2.1]
a.append("bilal")
print(a)
```

Result:



Task 2 Modifying List Elements via Indexing

```
b = [10, 20, 30]
b[1] = 99
print(b)
```

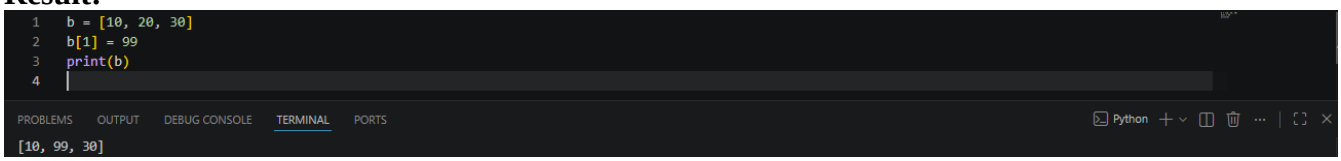
Result:



Task 3: Tuple Creation and Immutable Data Access

```
student = ("Ali", 18, "Computer Science")
print(student[0])
print(student[1])
```

Result:



Conclusions:

This lab successfully demonstrates the practical behavioral distinctions between **lists and tuples** in Python. While both store sequences of mixed data types and allow access via index values, lists offer dynamic structural flexibility through mutability and in-place methods, whereas tuples provide data protection and structure through forced immutabi

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab tasks	**Dated: 4 May 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 3**
Lab Title: Sets, Dictionary	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO3	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Week 8 LAB

Sets, Dictionary

Lab Goals/Objectives:

- To understand, create, and manipulate **Unordered Collections** in Python using Dictionaries and Sets.
- To master key-value mapping logic and dictionary mutability.
- To utilize set mathematical operations (unions, intersections, subsets) and enforce element uniqueness.

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

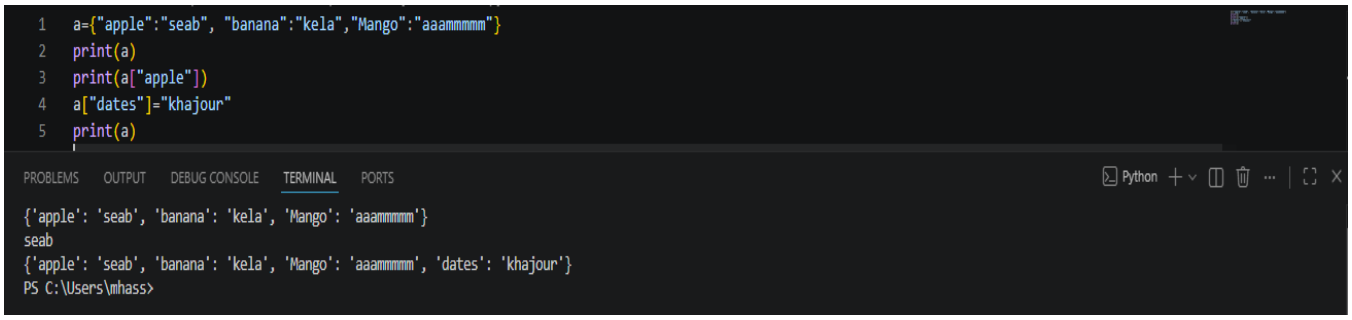
Python provides specialized built-in data types for managing unordered collections where positional indexing does not apply.

Lab Work:

Task 1: Dictionary Mapping, Access, and Key-Value Insertion

```
a = {"apple": "seab", "banana": "kela", "Mango": "aaammmmm"}
print(a["apple"])
a["dates"] = "khajour"
```

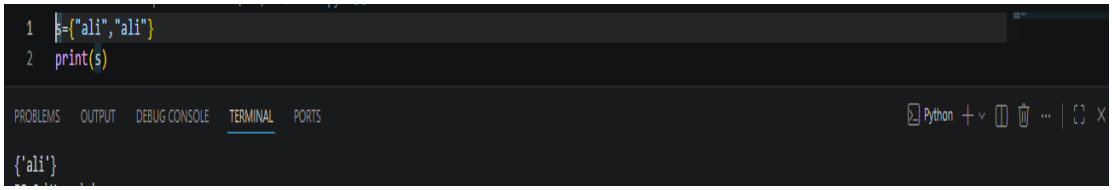
Result:



Task 2: Enforcing Set Uniqueness (Duplicate Elimination)

```
s = {"ali", "ali"}
print(s)
```

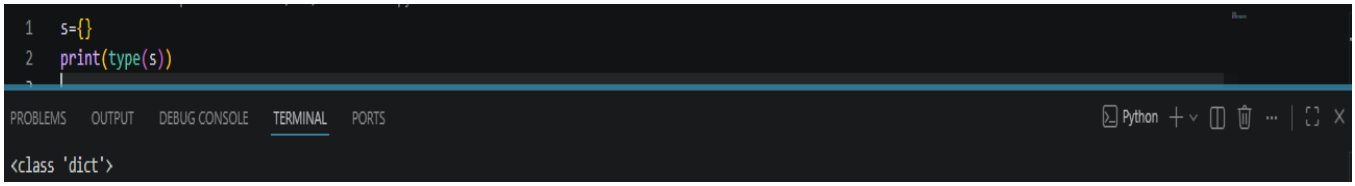
Result:



Task 3: Evaluating Default Empty Braces Type (`dict`).

```
s = {}
print(type(s))
```

Result:



Task 4: Explicit Initialization of an Empty Set.

```
s = set()
print(type(s))
```

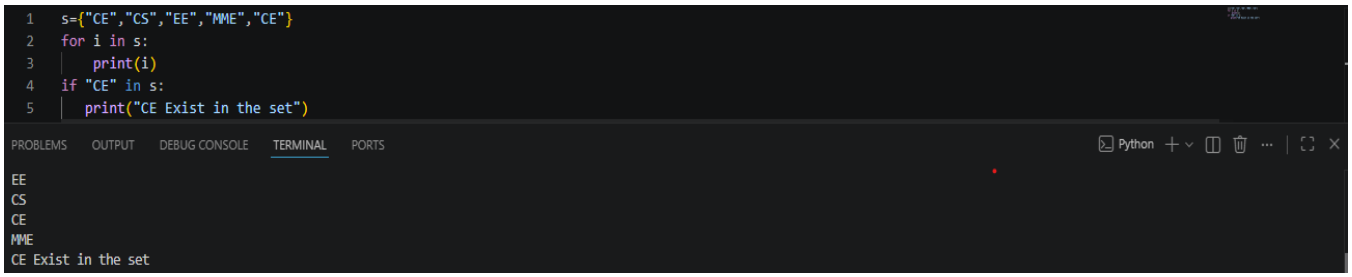
Result:



Task 5: Iterating Over Sets and Conditional Membership Membership Testing

```
for i in s:
  print(i)
if "CE" in s:
  print("CE Exist in the set")
```

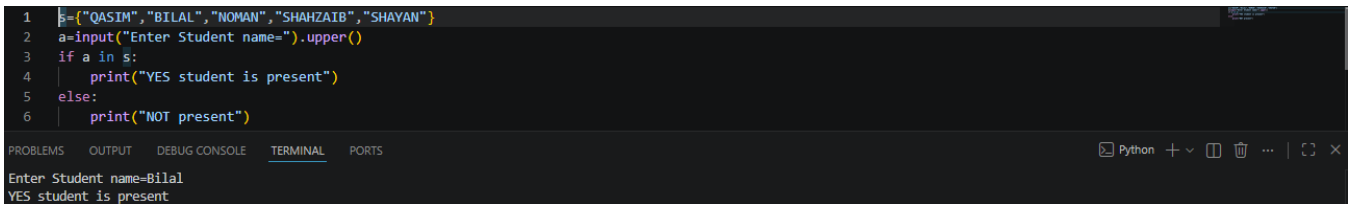
Result:



Task 6: Case-Insensitive String Matching using Set Lookups

```
a = input("Enter Student name=").upper()
if a in s:
  print("YES student is present")
```

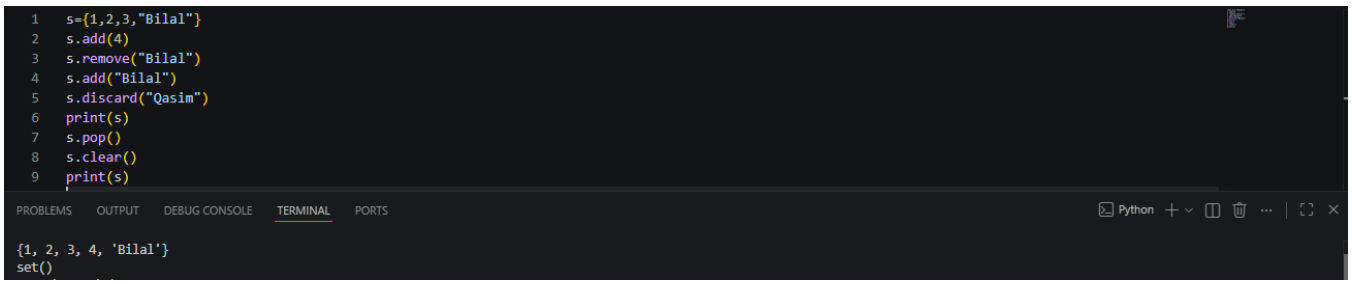
Result:



Task 7: Practical Testing of Mutating Set Methods

```
s.add(4)
s.remove("Bilal")
s.discard("Qasim")
s.pop()
s.clear()
```

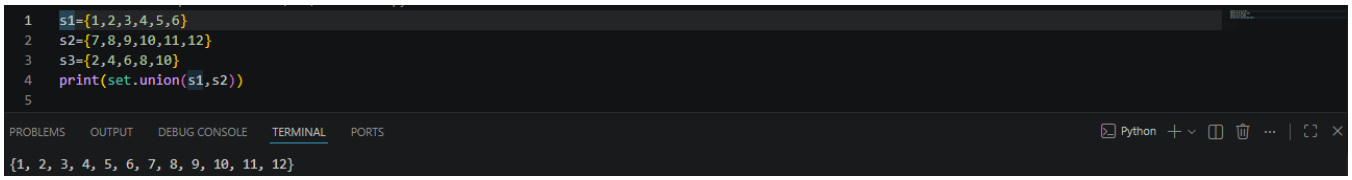
Result:



Task 8: Standard Set Union Operation

```
print(set.union(s1, s2))
```

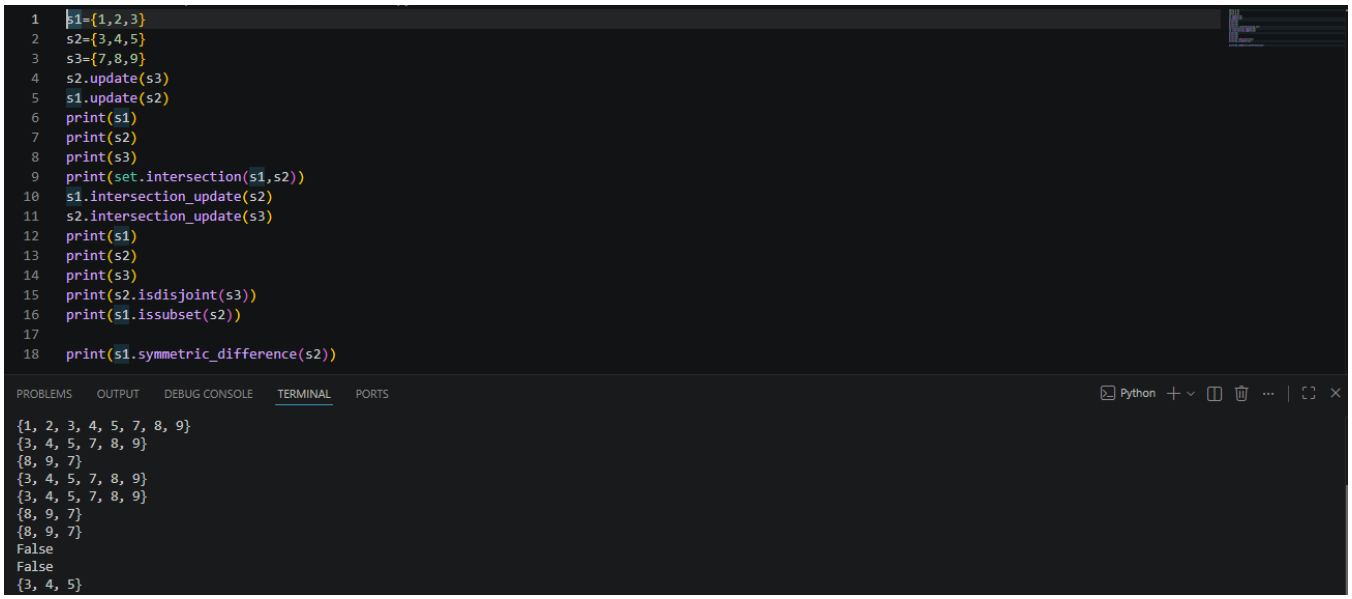
Result:



Task 9: Advanced Set Algebra (Intersections, Disjoint Checks, Subsets, and Symmetric Differences)

```
s2.update(s3)
print(set.intersection(s1, s2))
s1.intersection_update(s2)
print(s2.isdisjoint(s3))
print(s1.issubset(s2))
print(s1.symmetric_difference(s2))
```

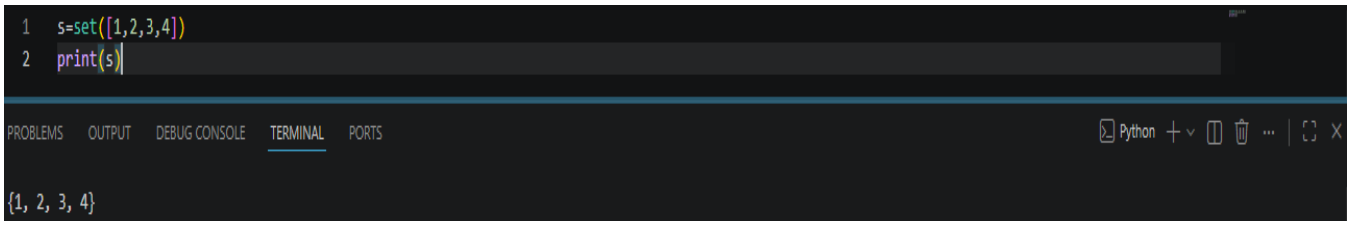
Result:



Task 10: Type Casting an Ordered List into an Unordered Set

```
s = set([1, 2, 3, 4])
print(s)
```

Result:



Conclusions:

This lab successfully demonstrates the behaviors of **hashed, unordered structures** in Python. We explored dictionaries as efficient key-value relational maps and sets as powerful mathematical tools. By working through uniqueness constraints, set mutations, and relational operations (like intersections and subsets), we established how to manage collection data where element identity and membership override numerical index positions.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab tasks	**Dated: 18 May 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 2**
Lab Title: Recursion	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO2	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Week 10 LAB

Recursion

Lab Goals/Objectives:

- To understand the core mechanism of **recursion** (functions that call themselves) in Python.

- To identify and implement the two essential components of a recursive function: the **base case** and the **recursive step**.

- To apply recursive logic to classic algorithmic problems, including countdowns, factorials, and sequence generation.

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

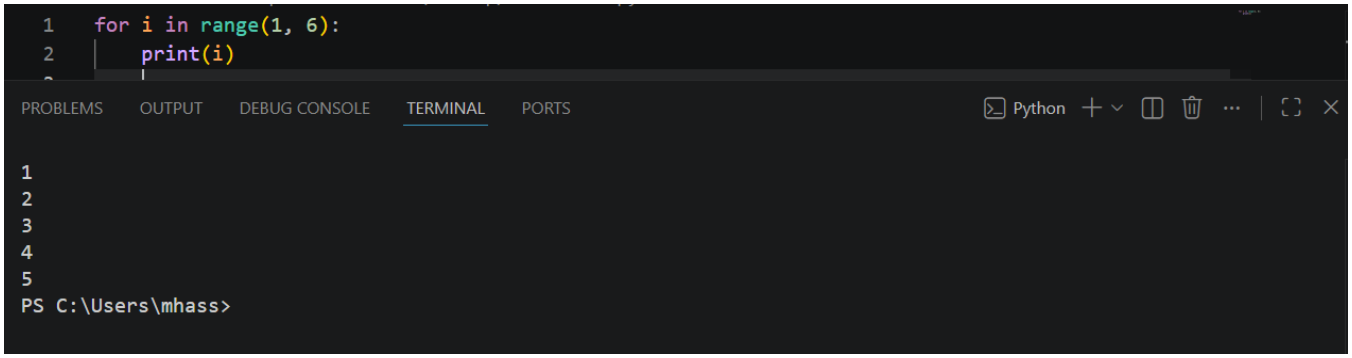
Recursion is a programming technique where a function solves a problem by breaking it down into smaller sub-problems of the same type, calling itself to solve them.

Lab Work:

Task 1: Recursive Countdown (Direct State Reduction)

```
def show(i):
        if (i==0):
            return
         print(i)
         show(i-1)
```

Result:



Task 2: Factorial Computation (Accumulative Call Stack Stack).

```
def show(n):
   if n==0 or n==1:
          return 1
   else:
          return n*show(n-1)
```

Result:

```
1  marks = [80, 90, 75, 60, 85]
2  total = 0
3  for m in marks:
4      total = total + m
5  print("Total marks:", total)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS        Python

Total marks: 390
PS C:\Users\mhass>
```

Task 3: The Fibonacci Sequence (Multiple Branches Tree Recursion).

```
def fibonacci_series(n):
    if n <= 0:
      return "Input should be a positive integer."
    elif n == 1:
     return 0
    elif n == 2:
      return 1
    else:
      return fibonacci_series(n-1) + fibonacci_series(n-2)
```

Result:

```
1  for i in range(1, 11):
2      if i % 2 == 0:
3          print(i)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS        Python

2
4
6
8
10
PS C:\Users\mhass>
```

Conclusions:

This lab successfully demonstrates the implementation of **recursive structures** in Python. We explored how explicit base cases prevent stack overflows, how linear recursion can calculate sequential products like factorials, and how multiple branching recursion evaluates complex sequence dependencies like the Fibonacci series.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab Tasks	**Dated: 18 May 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 4**
Lab Title: Exception Handling	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO4	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 1

Exception Handling

Lab Goals/Objectives:

- To understand the purpose and execution flow of **Exception Handling** in Python.
- To learn how to intercept runtime errors using `try` and `except` blocks to prevent programs from crashing.
- To handle unexpected user input safely and ensure remaining application code continues executing.

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

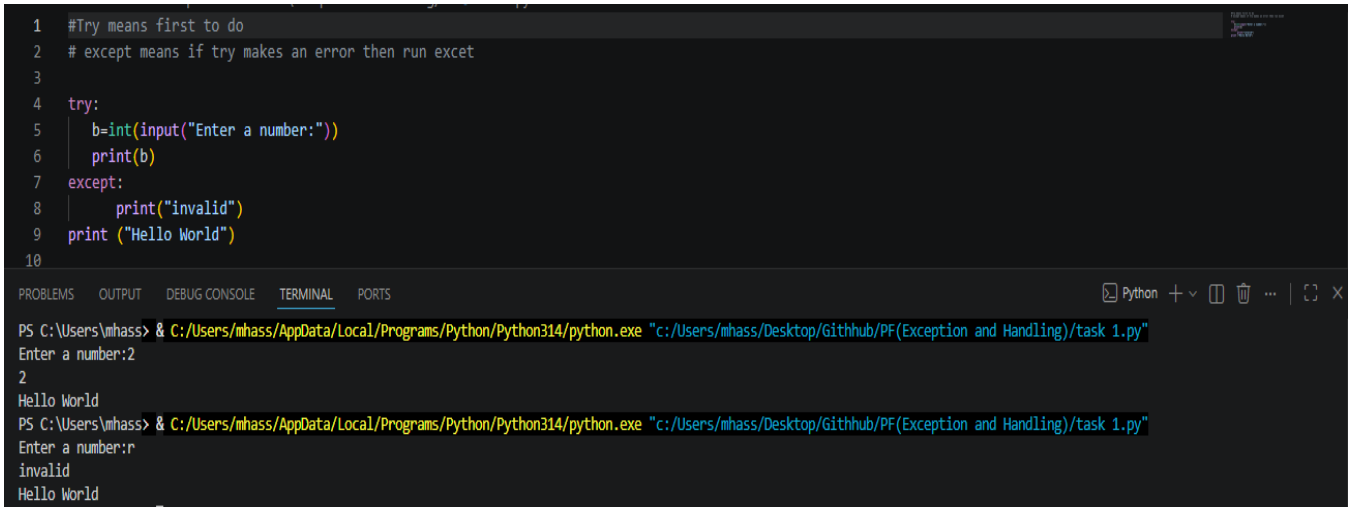
An exception is an error that occurs during the execution of a program. If left unhandled, an exception immediately halts the script and displays a traceback error.

Lab Work:

Task 1: Handling Invalid Type Casting Input

```
try:
        b = int(input("Enter a number:"))
        print(b)
except:
        print("invalid")
print("Hello World")
```

Result:



Conclusions:

This lab successfully demonstrates the implementation of defensive programming using **Exception Handling** in Python. By wrapping unpredictable operations—such as user-driven type casting—inside a `try-except` structure, we observed how to prevent unexpected string inputs from causing a script crash, thereby keeping the trailing logic safe and functional.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab tasks	**Dated: 25 May 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 2**
Lab Title: Lambda Functions	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO2	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Week 12 LAB

Lambda Functions

Lab Goals/Objectives:

- To understand the definition and purpose of **Anonymous Functions (Lambda)** in Python.
- To master the syntactic differences between standard named functions (`def`) and inline expressions.

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

Python supports the creation of anonymous functions—functions defined without a name—using the `lambda` keyword.

Lab Work:

Task 1: Single-Argument Lambda Expression (Squaring a Value)

```
a = lambda x: x * x
print(a(5))
```

Result:



Task 2: Multi-Argument Lambda Expression (Summing Values)

```
a = lambda x, y: x + y
print(a(2, 4))
```

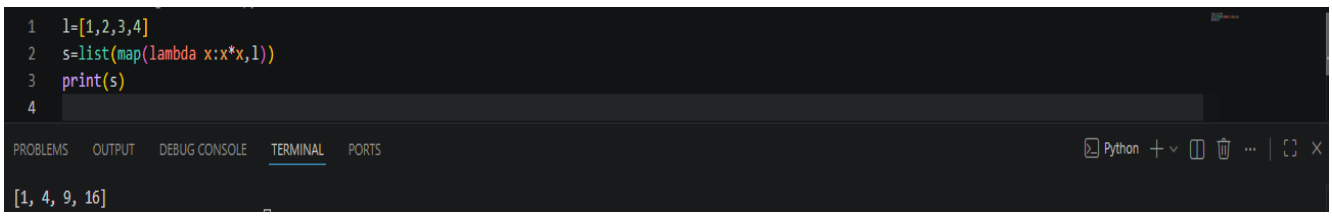
Result:



Task 3: Transforming Iterables with `map()` and Lambda.

```
l = [1, 2, 3, 4]
s = list(map(lambda x: x * x, l))
print(s)
```

Result:



Conclusions:

This lab successfully demonstrates the conciseness and utility of **lambda functions** in Python. We established that while lambda expressions are structurally restricted to single-line logic, they serve as highly effective tools for quick discardable operations, particularly when passing operational logic inline to higher-order sequence transformers like `map()`.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab tasks	**Dated: 25 May 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 2**
Lab Title: MAP, Filter and Reduce	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO2	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Week 13 LAB

MAP, Filter and Reduce

Lab Goals/Objectives:

- To understand and apply **functional programming tools** (`map`, `filter`, and `reduce`) in Python.
- To practice passing anonymous **lambda functions** as operational logic into higher-order functions.
- To master transforming collections, extracting selective elements, and rolling up datasets into a single aggregate value.

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

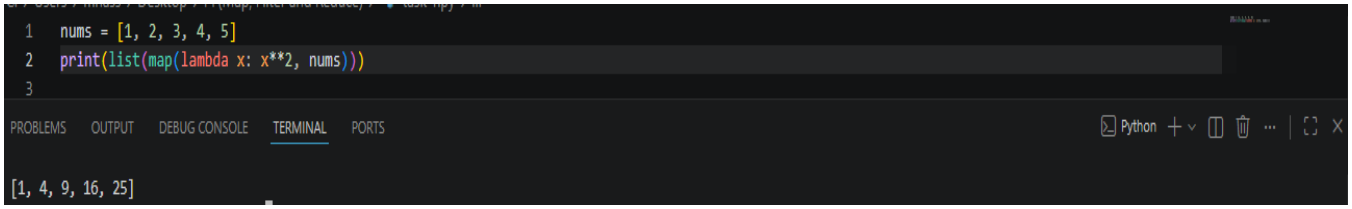
Python provides a powerful suite of higher-order functions designed to process iterables cleanly and efficiently without relying on explicit `for` or `while` loops.

Lab Work:

Task 1: Transforming Numbers (`map()` and Squaring)

```
nums = [1, 2, 3, 4, 5]
print(list(map(lambda x: x**2, nums)))
```

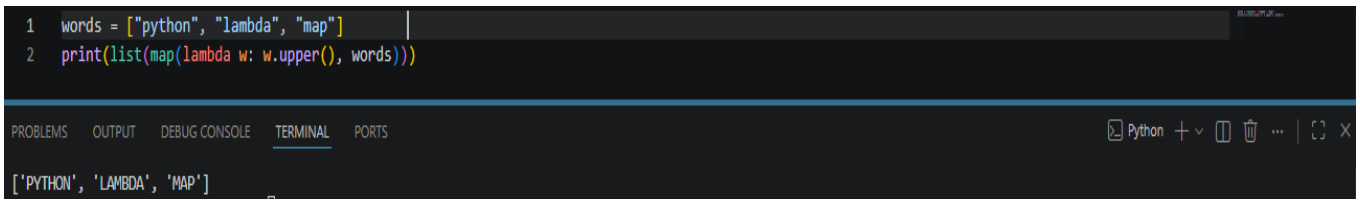
Result:



Task 2: Transforming Strings (`map()` and Capitalization)

```
words = ["python", "lambda", "map"]
print(list(map(lambda w: w.upper(), words)))
```

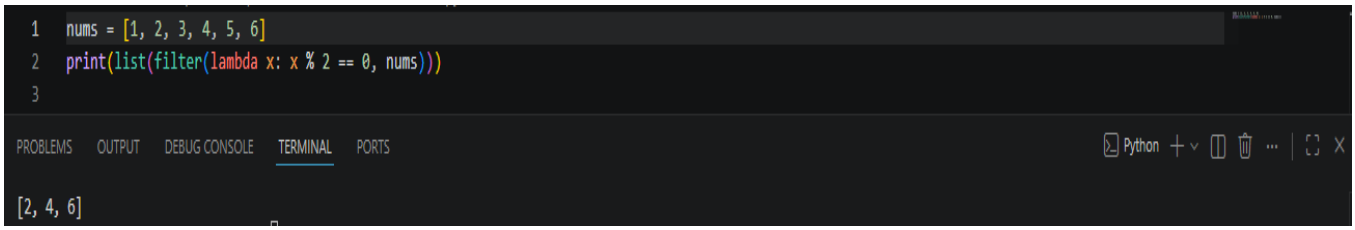
Result:



Task 3: Selective Extraction (`filter()` and Even Numbers)

```
nums = [1, 2, 3, 4, 5, 6]
print(list(filter(lambda x: x % 2 == 0, nums)))
```

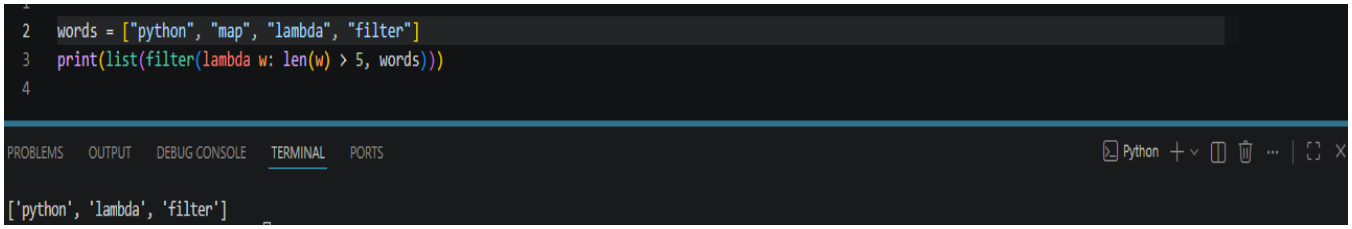
Result:



Task 4: Content Filtering (`filter()` and String Length).

```
words = ["python", "map", "lambda", "filter"]
print(list(filter(lambda w: len(w) > 5, words)))
```

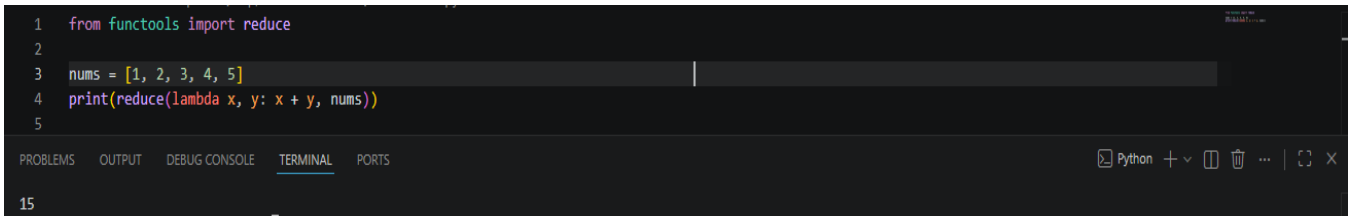
Result:



Task 5: Sequence Aggregation via Summation (`reduce()`).

```
nums = [1, 2, 3, 4, 5]
print(reduce(lambda x, y: x + y, nums))
```

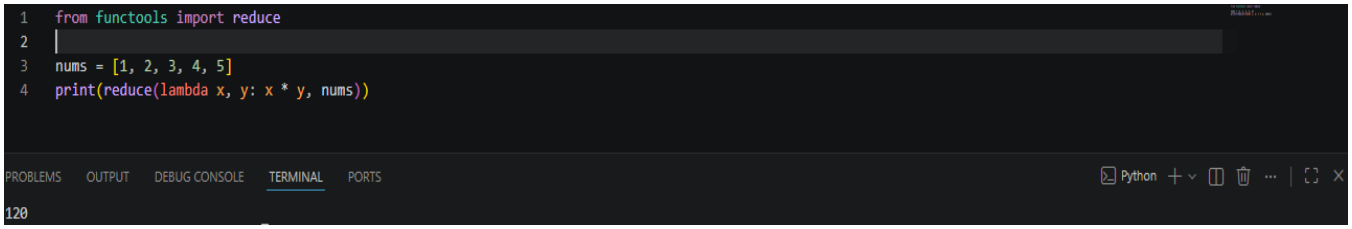
Result:



Task 6: Sequence Aggregation via Multiplication (`reduce()`).

```
nums = [1, 2, 3, 4, 5]
print(reduce(lambda x, y: x * y, nums))
```

Result:



Conclusions:

This lab successfully demonstrates the core capabilities of **functional array processing pipelines** in Python. By mapping over items to alter state, filtering collections to extract specific criteria, and reducing sequences to aggregate summaries, we demonstrated how complex looping algorithms can be cleanly written using expressive, single-line lambda functions.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab tasks	**Dated: 1 June 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 2**
Lab Title: OS Library, if __name__ == "__main__", is vs ==	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO2	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Week 2 LAB

OS Library, if __name__ == "__main__", is vs ==

Lab Goals/Objectives:

- To understand the concept of **modular programming** by splitting code into separate Python files.
- To learn how to reference and execute code from external files using the `import` statement.
- To master the purpose of the special `__name__ == "__main__"` conditional check.

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

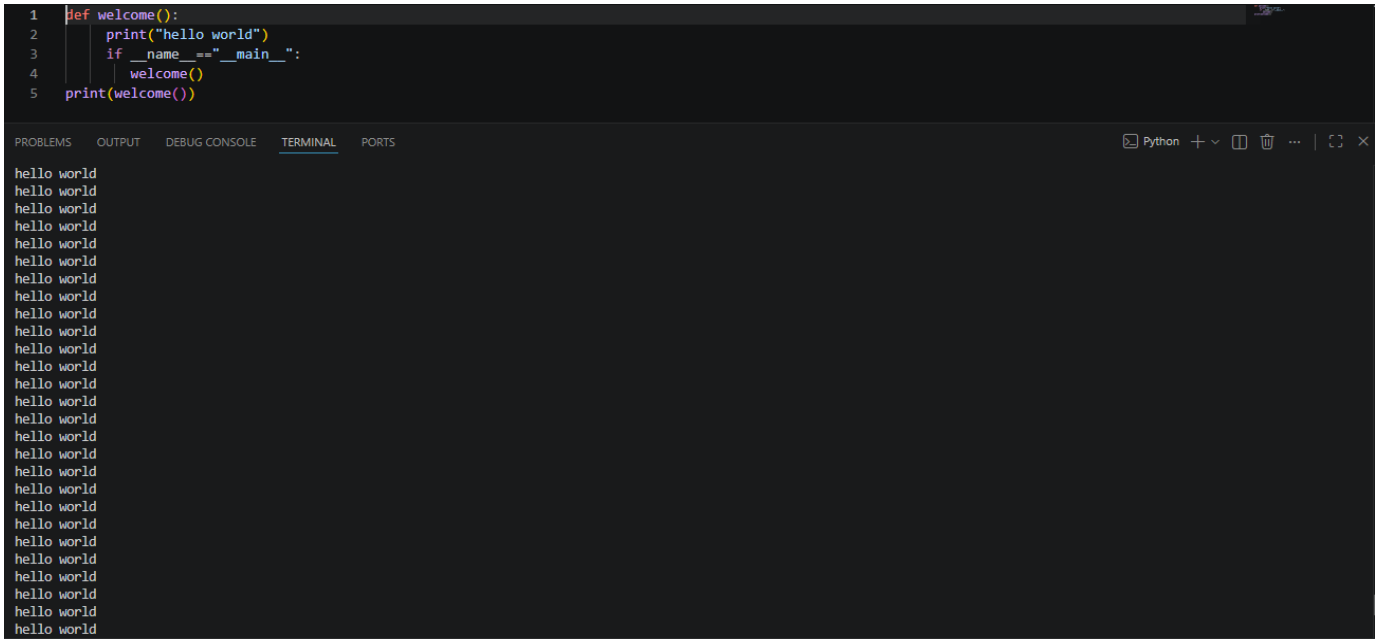
As programs grow larger, keeping all code in a single file becomes unmanageable. Python allows you to break scripts down into separate reusable files called **modules**.

Lab Work:

Task 1: Defining a Reusable Module Function

```
def welcome():
        print("hello world")
        if __name__=="__main__":
                welcome()
print(welcome())
```

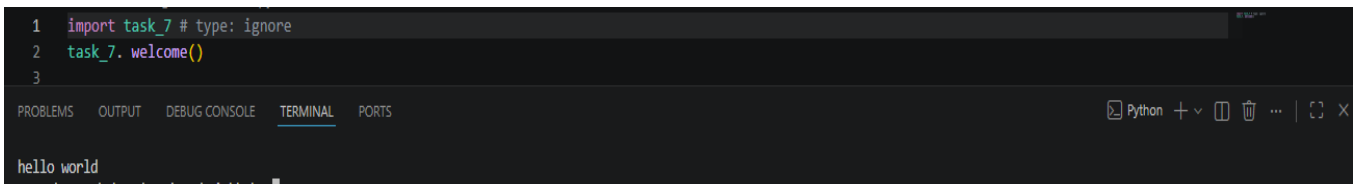
Result:



Task 2: Importing and Invoking External Module Logic

```
import task_7
task_7.welcome()
```

Result:



Conclusions:

This lab successfully demonstrates the fundamentals of **code serialization and modularity** in Python. We explored how to build independent code assets in a module file and call them from an external script. Furthermore, we analyzed how the `__name__` variable acts as an execution guard to separate immediate standalone test scripts from safe, importable function collections.

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Lab Tasks	**Dated: 7 June 2026**
Semester: First	**Session: 2026 Spring**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO 3**
Lab Title: GUI using Qt Designer	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO3	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Week 15 LAB

GUI using Qt Designer

Lab Goals/Objectives:

- To understand and design Graphical User Interfaces (GUIs) using **Qt Designer (`.ui` layout XMLs)**.
- To master structural widget organization using layouts (`QVBoxLayout`, `QHBoxLayout`) and custom container spacing.
- To design intuitive multi-role workflows involving separate authentication screens, input entry forms, data visualizers, and administrative tables.

Equipment Required:

* Computer/Laptop
* Python 3.14
* VS Code or pycharm

Theory:

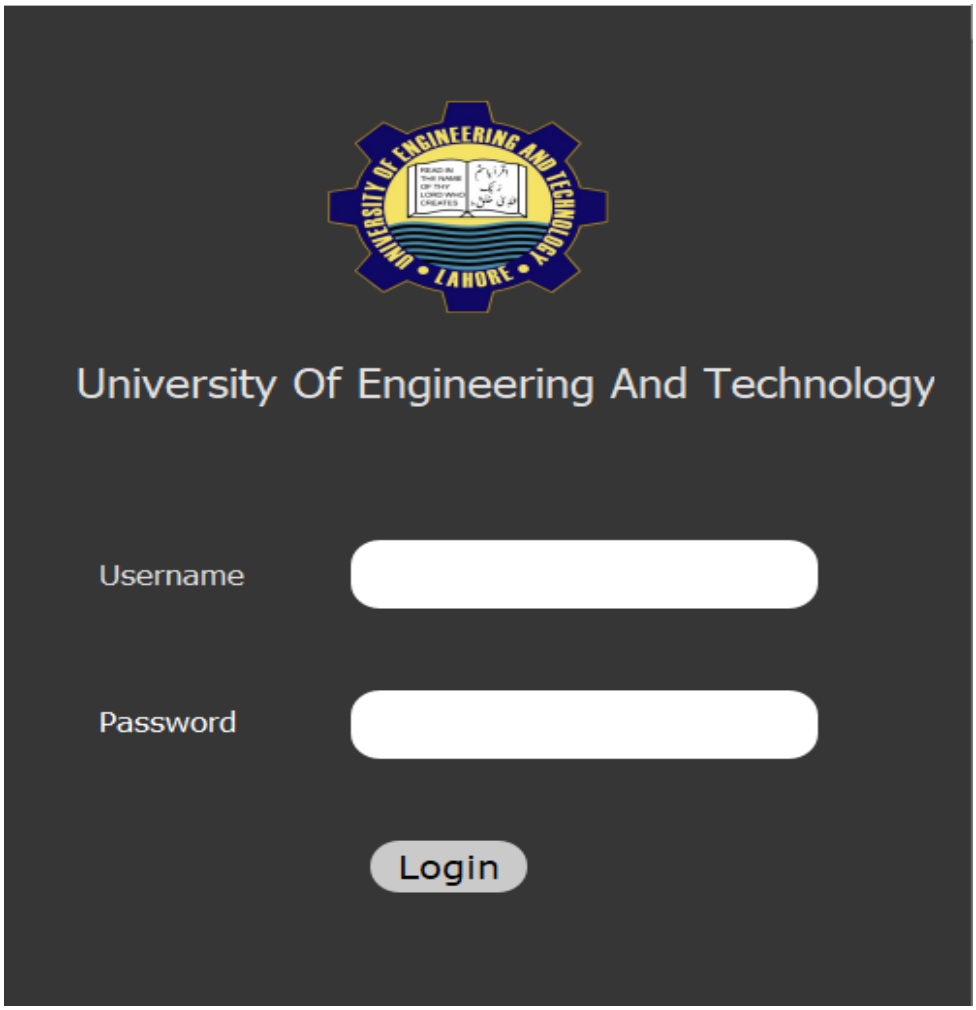
Qt Designer is a visual layout tool used to design cross-platform user interfaces. The resulting `.ui` file is a structured XML document describing widget hierarchies, geometries, and visual styles.

Lab Work:

Task 1: Institutional Portal Interface (`UET login Page.ui`)

- **Core Components:** Dark contrast wrapper background (`rgb(54, 54, 54)`), explicit single-line credential forms (`QLineEdit`), rounded edges (`border-radius:15px`), and an institutional banner label.
- **Design Focus:** Simple, absolute coordinate layouts emphasizing clean readability and basic entry constraints.

Result:

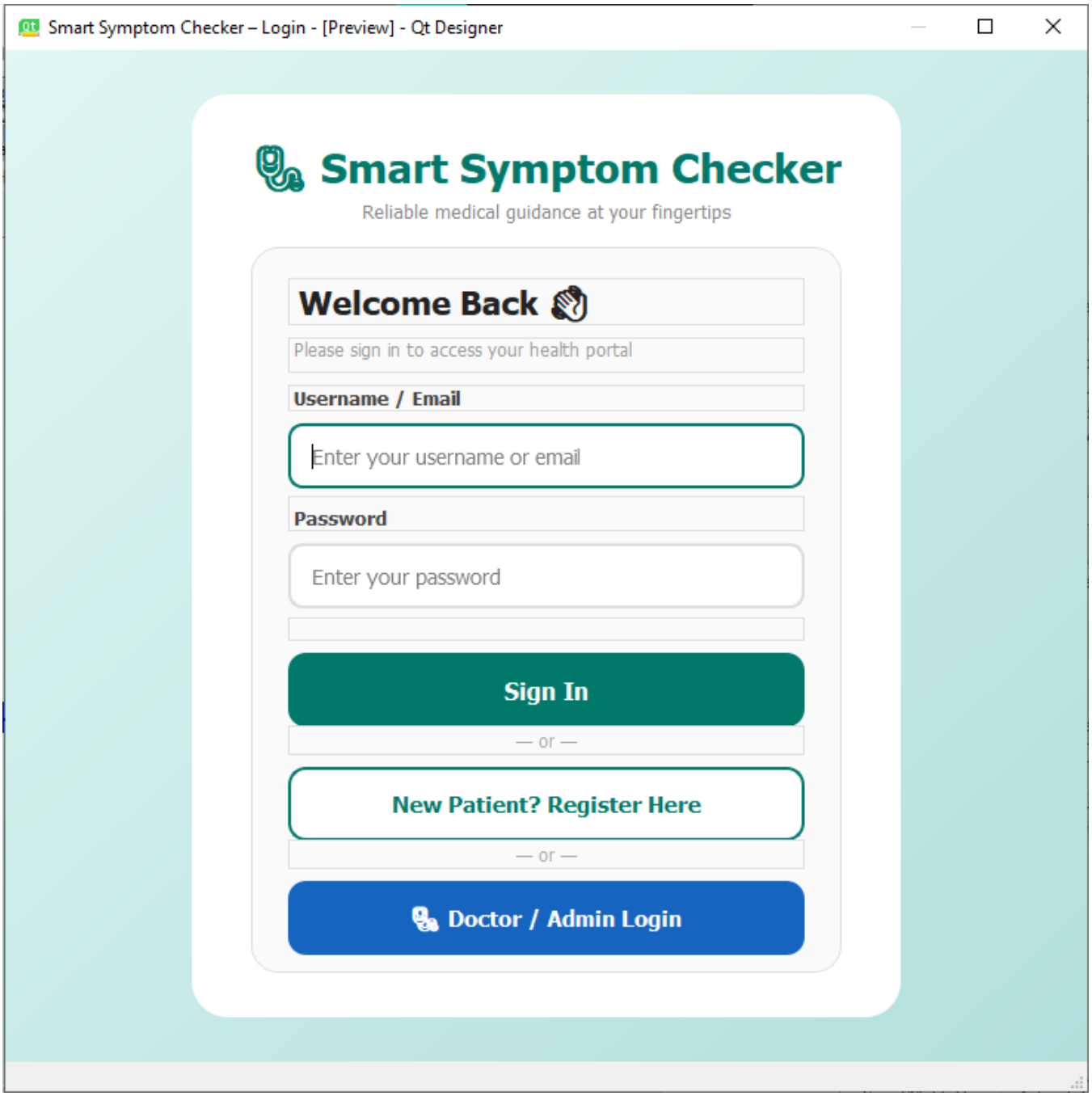


Task 2: Multi-Role Patient Authentication (`login.ui`)

- **Core Components:** Custom linear gradients (`stop:0 #e0f7f4, stop:1 #b2dfdb`), centered multi-input card alignment fields, and a distinct alternate login redirect option (`doctorLoginButton`).

- **Design Focus:** Aesthetic entry gates that cleanly segment user privileges right from the access layer.

Result:

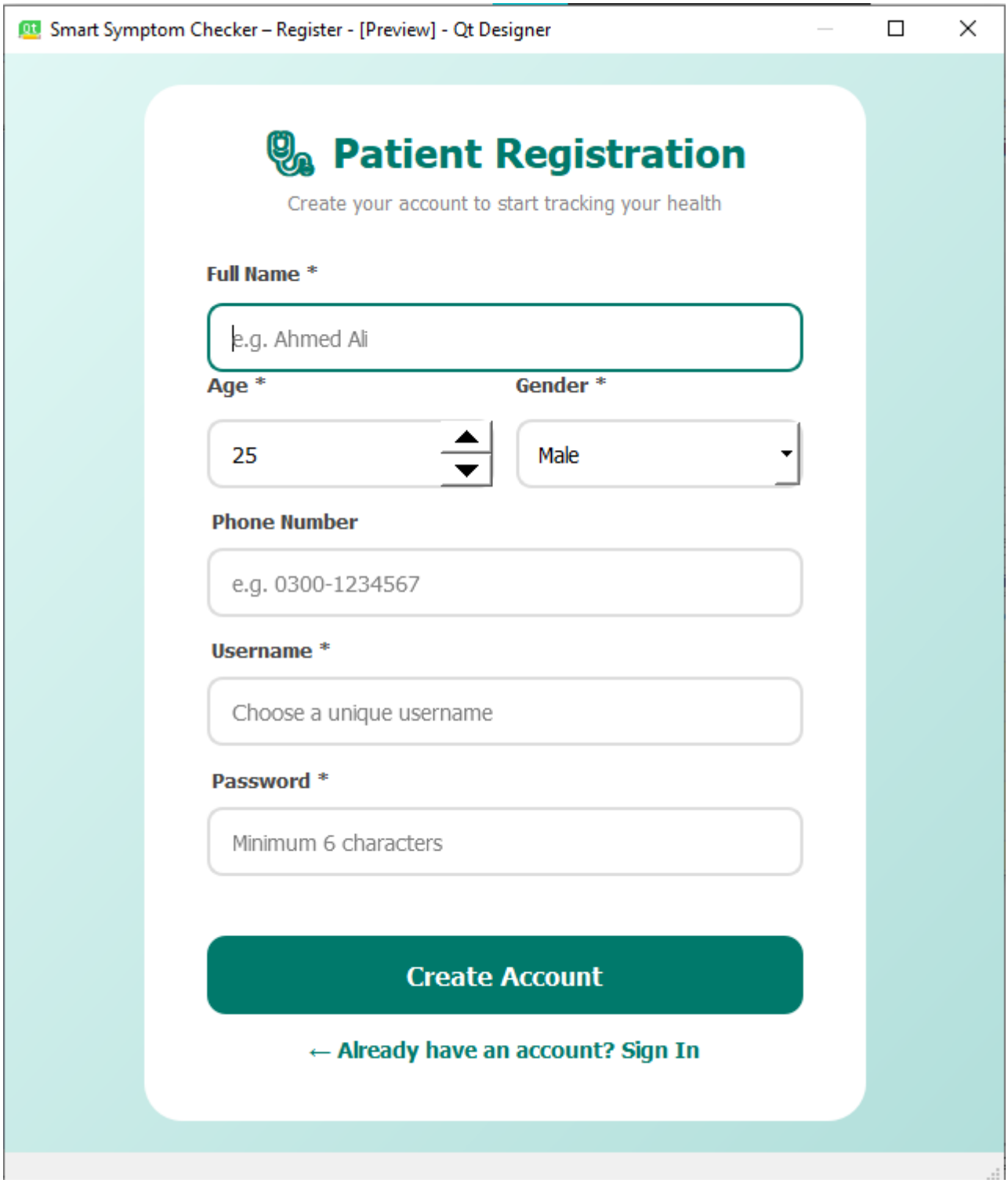


Task 3: Interactive Account Creation (`register.ui`)

- **Core Components:** Extended input submission layout fields, validation inputs, custom active submission blocks (`registerButton` with solid teal background `#00796B`), and a login fallback route button.

- **Design Focus:** Complete input validation structures ensuring smooth user registration pipelines.

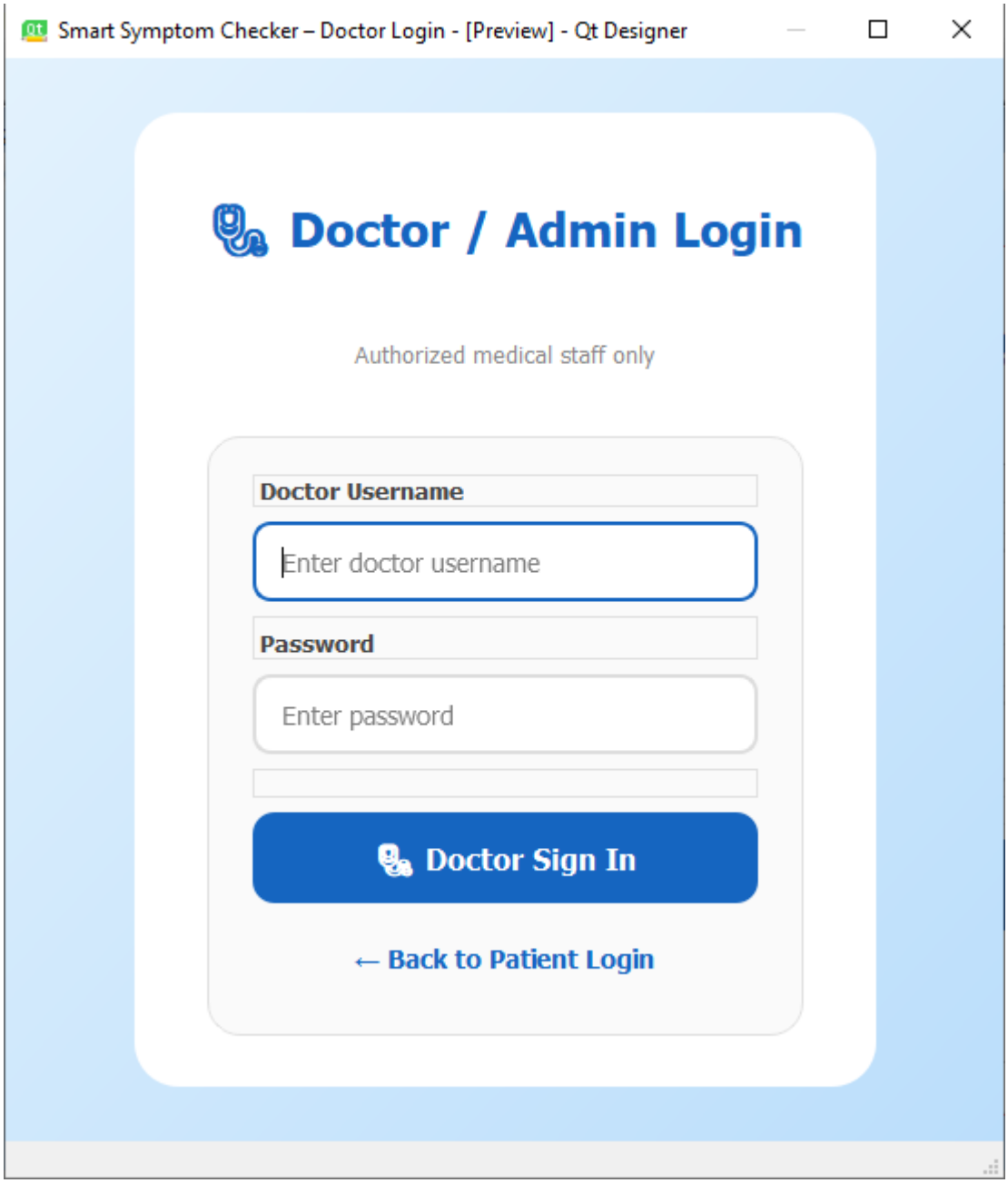
Result:



Task 4: Privileged Access Entry Portal (`doctor_login.ui`)

- **Core Components:** Secure credential masking text boxes, high-visibility interactive signature buttons (□ `Doctor Sign In` utilizing rich hovering effects), and standard back-navigation triggers.
- **Design Focus:** Dedicated secondary authentication screen designed for secure clinical administrator sessions.

Result:

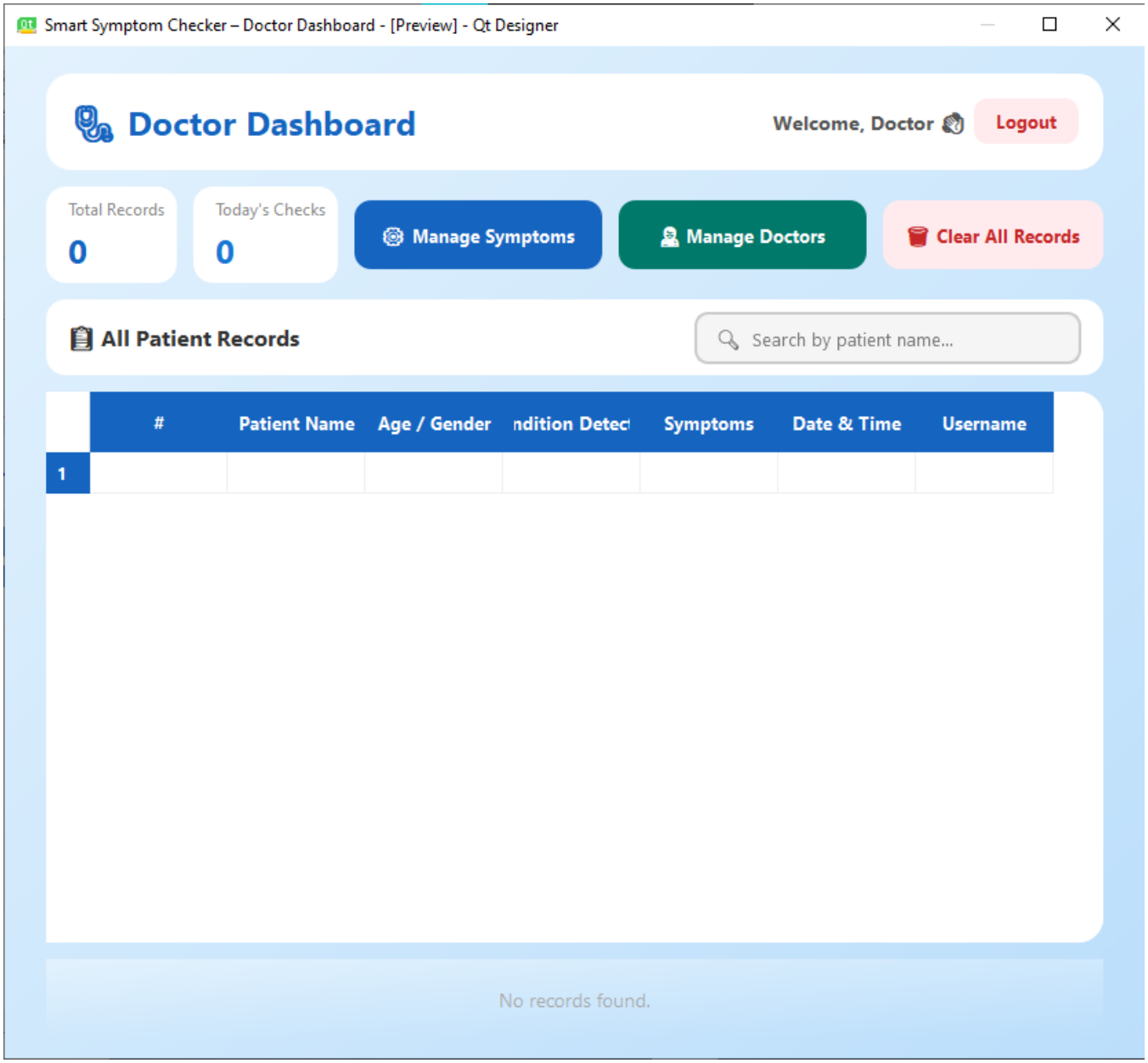


Task 5: Clinical Operation Hub (`doctor_dashboard.ui`)

- **Core Components:** Multi-column historical tracking layout table (`QTableWidget`), alternating color row parameters, and dynamic status labels (`emptyLabel`) to handle blank results elegantly.

- **Design Focus:** Creating a high-density, analytical data control deck tailored for medical practitioners.

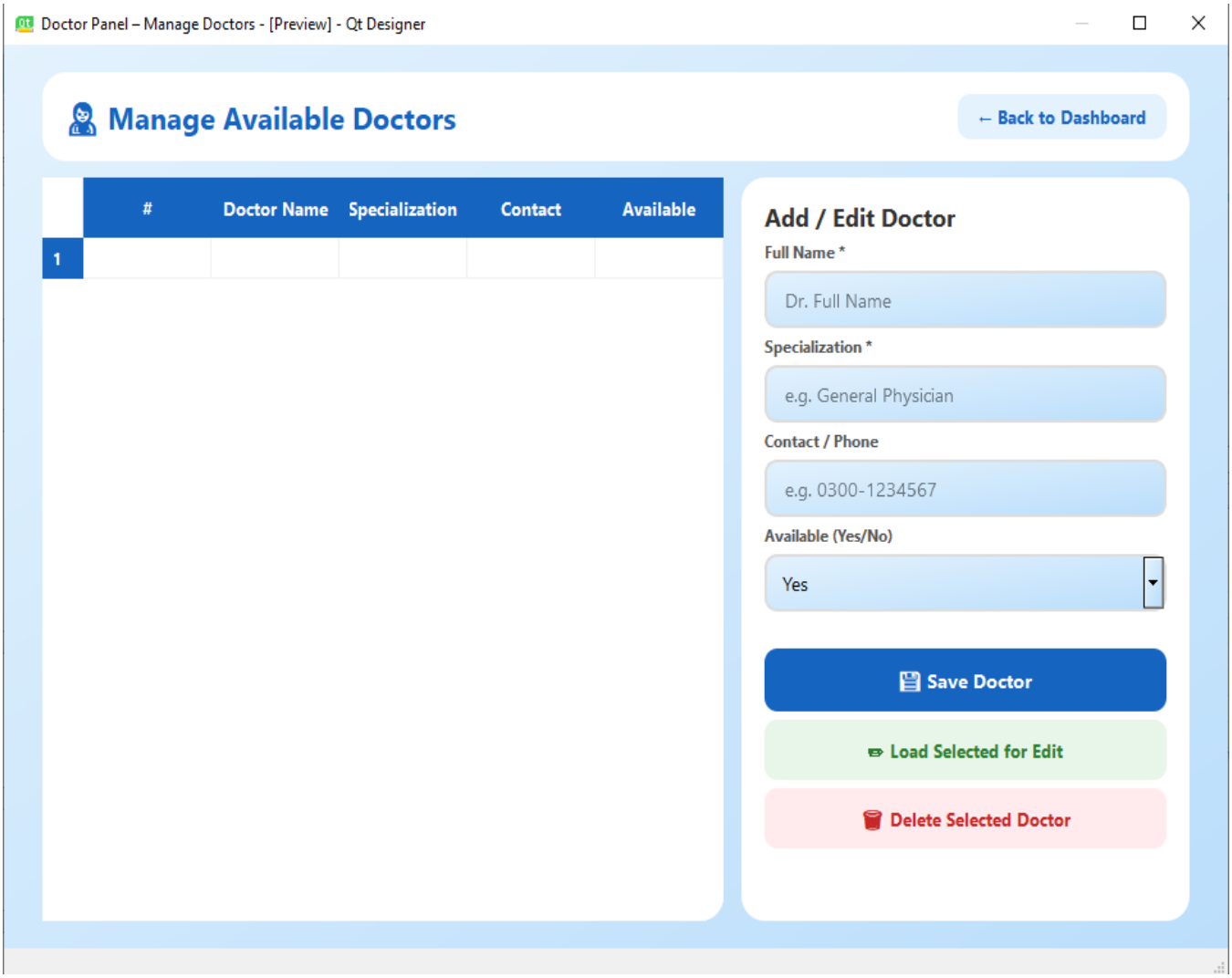
Result:



Task 6: Directory Management System (`manage_doctors.ui`)

- **Core Components:** Database row interaction controls, custom modifying triggers (`editDocButton`), explicit deletion actions (`deleteDocButton` with distinct caution-red styling `#c62828`), and spacer configurations.
- **Design Focus:** Standard CRUD (Create, Read, Update, Delete) administrative control flows.

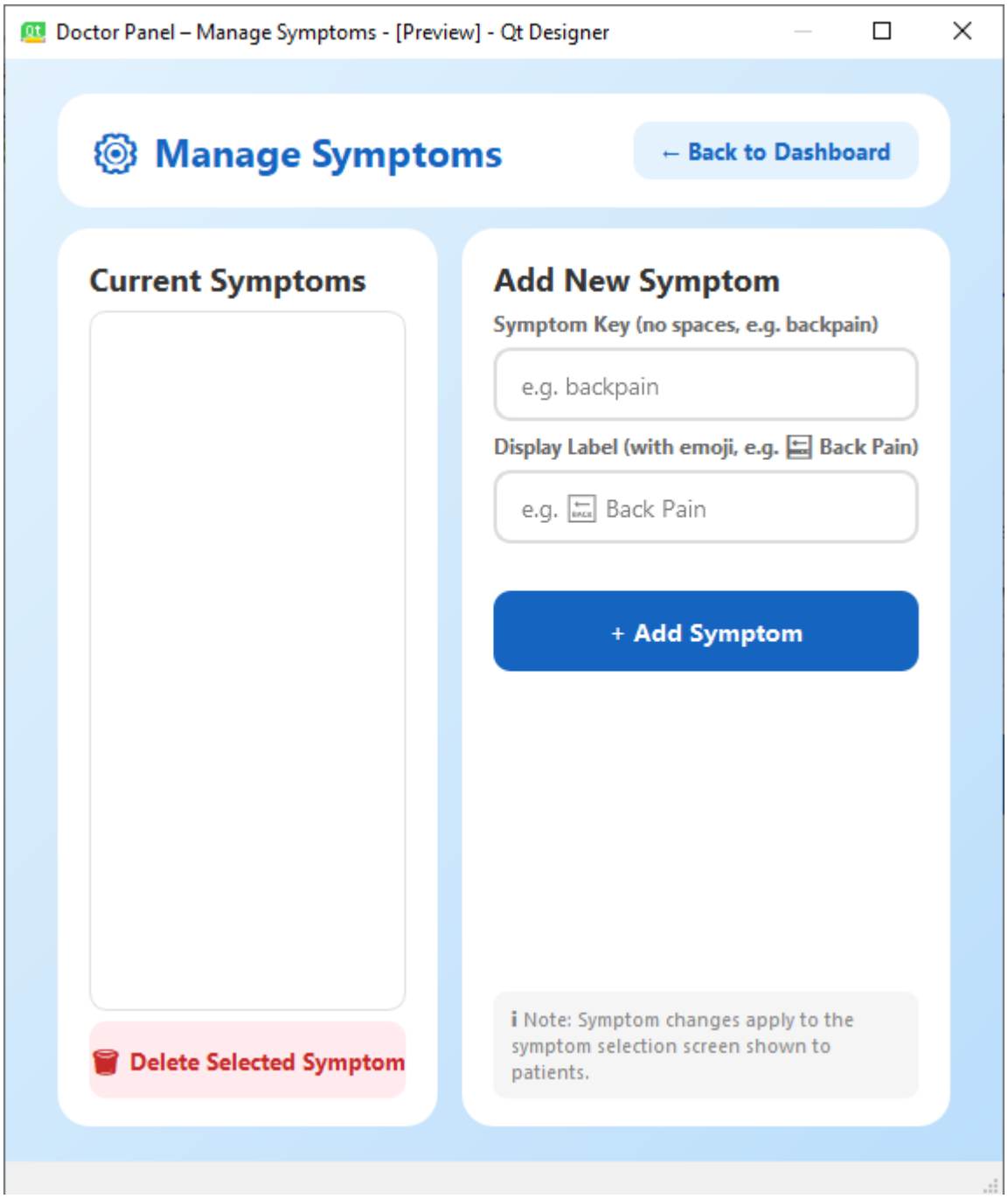
Result:



Task 7: Dynamic Parameter Customization Interface (`manage_symptoms.ui`)

- **Core Components:** Custom inline addition handlers (`addSymptomButton`), input mapping grids, and informative hint containers styled for persistent user documentation.
- **Design Focus:** Allowing system administrators to safely modify variables that directly alter front-end user experiences.

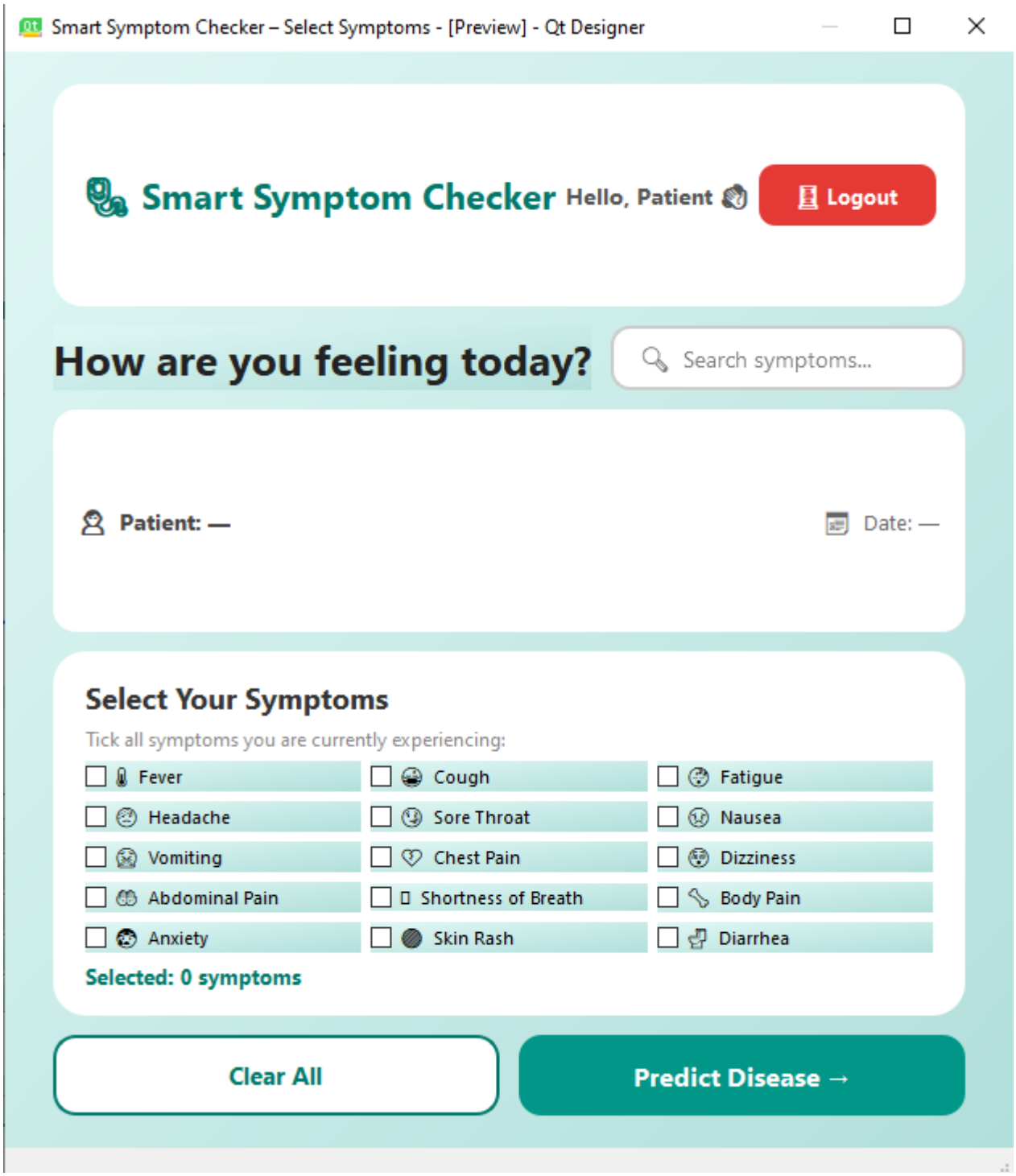
Result:



Task 8: Medical Feature Selection Questionnaire (`symptoms.ui`)

- **Core Components:** Large element layout arrangements, multiple selection choice boxes, reset state triggers (`clearButton`), and distinct action execution prompts (`Predict Disease →`).
- **Design Focus:** Optimizing multi-input check patterns to gather clinical parameters comfortably.

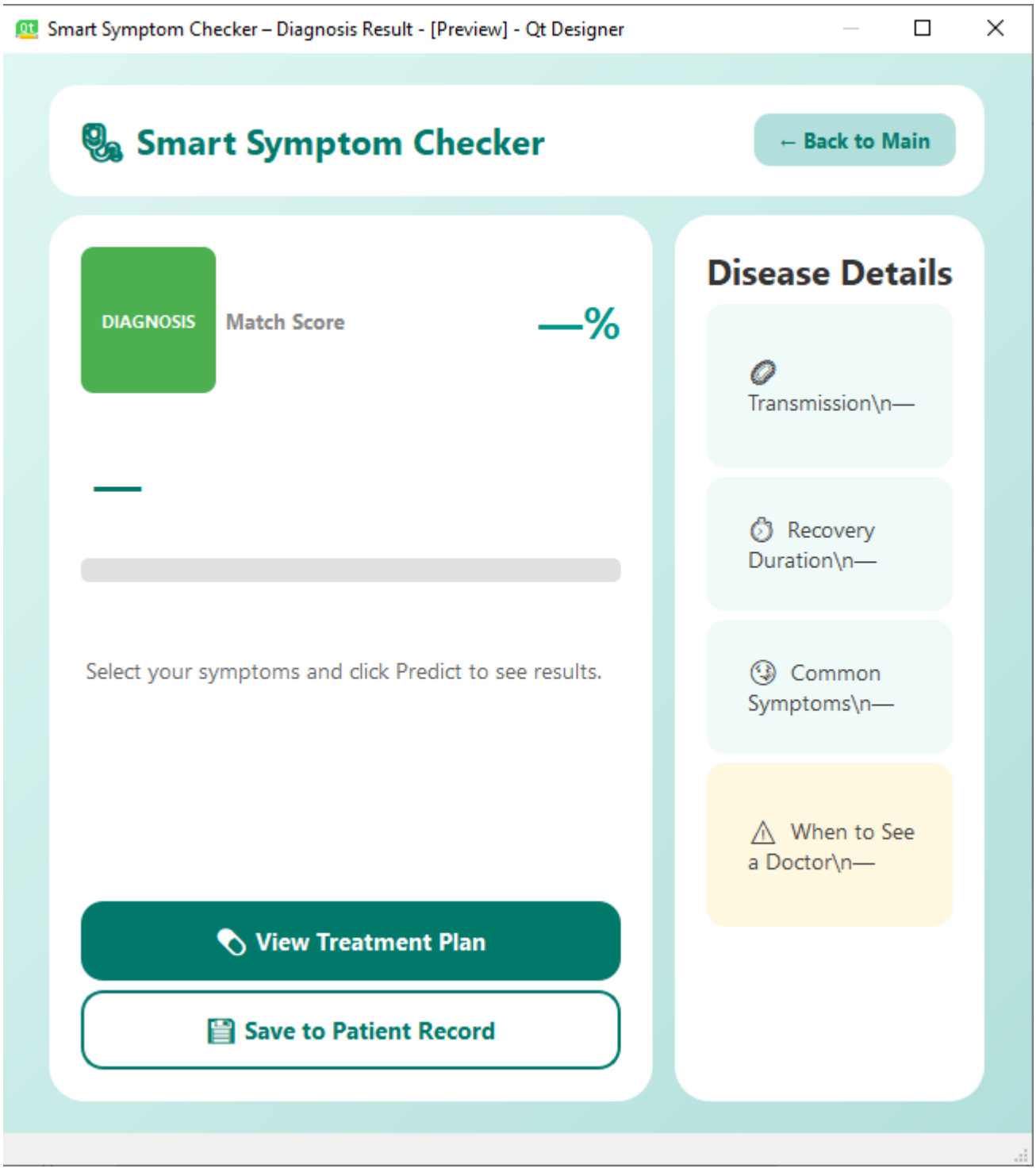
Result:



Task 9: Diagnostic Evaluation Feedback Screen (`results.ui`)

- **Core Components:** Informational panel layouts (`infoCard4`), styled bullet labels (`warningLabel` colored in amber `#fff8e1`), and prominent condition results displays.
- **Design Focus:** Presenting hierarchical information effectively by keeping classification outputs separate from general notes.

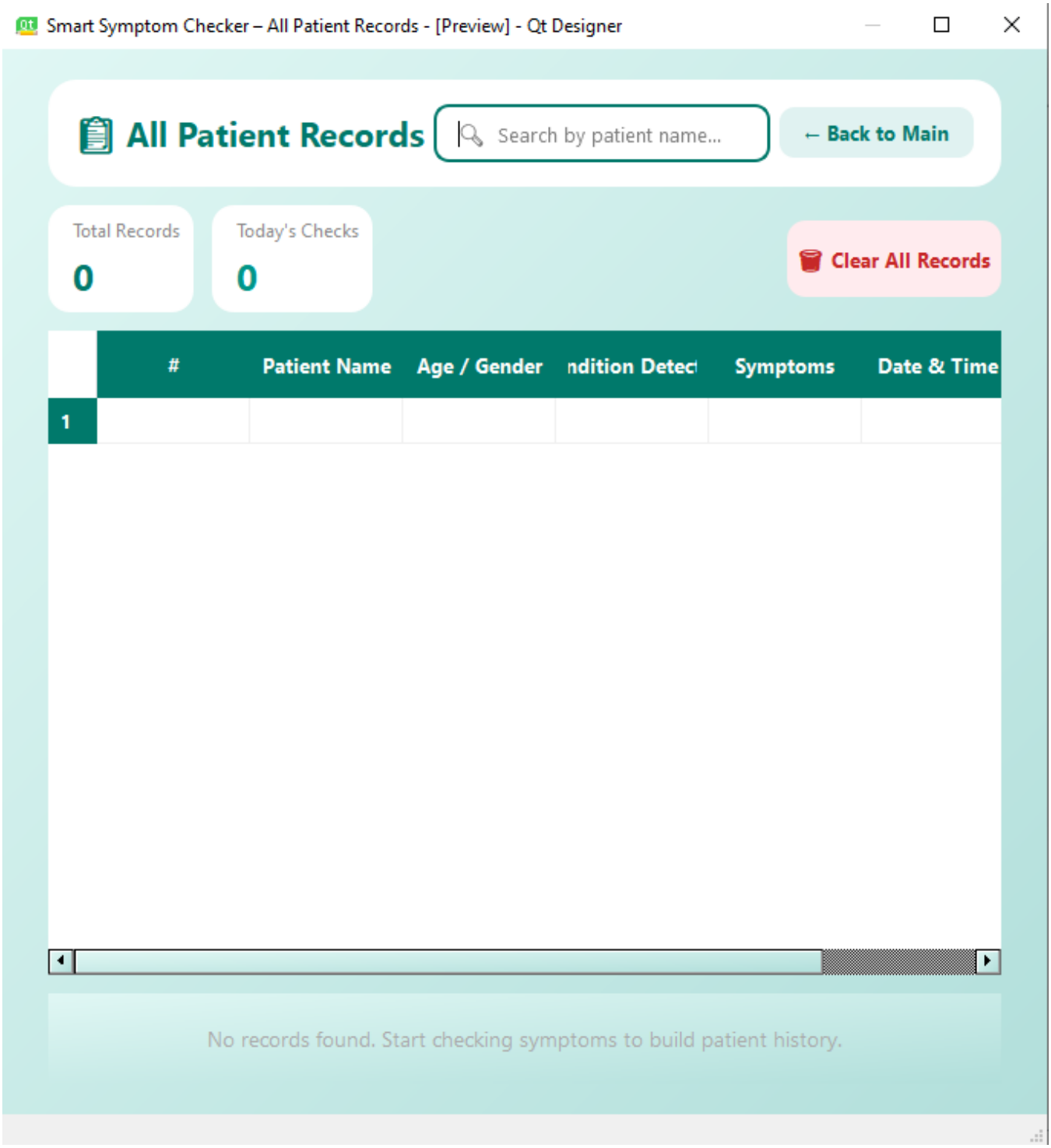
Result:



Task 10: Complete Patient History Log (`records.ui`)

- **Core Components:** Wide tabular arrays (`QTableWidget`) detailing timestamps, demographics, and symptom outputs alongside automated placeholder reminders.
- **Design Focus:** Full log aggregation ensuring seamless navigation through archived data assets.

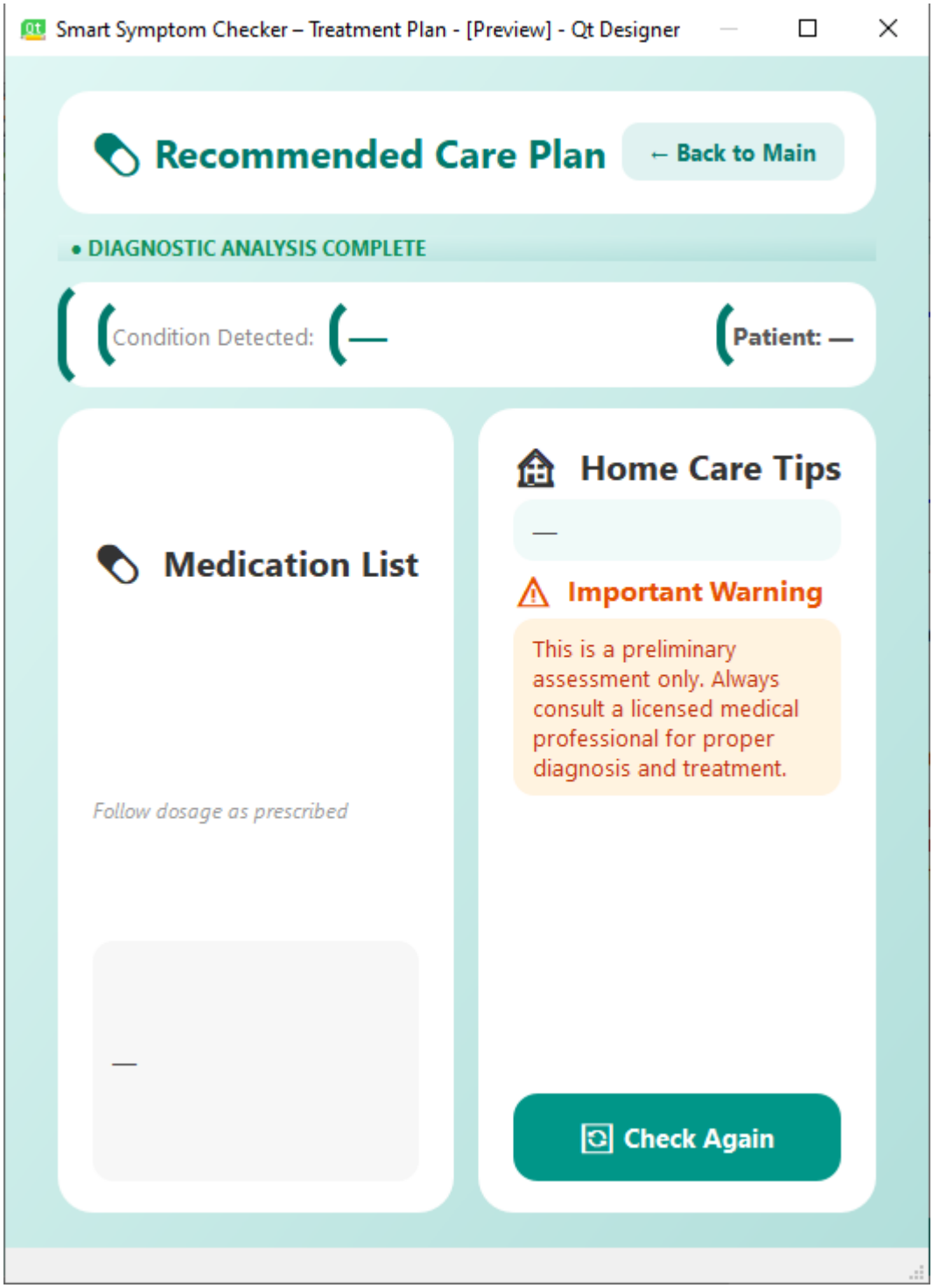
Result:



Task 11: Clinical Recovery Plan Screen (`treatment.ui`)

- **Core Components:** Main description display fields (`treatmentBox`), safety disclaimer containers (`warningBoxText`), standalone warning text fields, and a prominent loop-back trigger button (`checkAgainButton`).
- **Design Focus:** Providing clean, separate sections for final therapeutic feedback while ensuring the user can easily find the button to restart the questionnaire workflow safely.

Results:



Conclusions:

This lab successfully demonstrates how **Qt XML layouts** standardize front-end designs for desktop applications. By mapping distinct interfaces—ranging from entry portals to complex data tables—we established how structured styling sheets, vertical/horizontal layouts, and clean widget hierarchies combine to form modern, adaptive, and highly functional user systems.